國立中央大學

114 學年度 第一學期

軟體工程 期末書面報告

D 物件設計文件

寵物領養平台

組別:第十七組

組員:

114423001 陳柏甫

114423007 潘維晟

114423042 劉和媛

114423043 李嶧

目錄

1. 文件概述

1.1 目的

本文件描述系統的詳細物件設計，包括： - 資料模型結構與關聯 - API 端點詳細規格（輸入/輸出 JSON 格式） - 關鍵 Use Case 的循序圖

1.2 適用範圍

此文件適用於開發團隊進行程式碼實作，包含前端（Vue 3 + TypeScript）與後端（Flask + Python）的物件層級設計。

---

2. 核心資料模型 (Data Models)

2.1 User (使用者)

檔案位置: backend/app/models/user.py

資料結構:

```
class UserRole(Enum):
    GENERAL_MEMBER = 'GENERAL_MEMBER'   # 一般會員
    SHELTER_MEMBER = 'SHELTER_MEMBER'   # 收容所會員
    ADMIN = 'ADMIN'                     # 管理員

class User(db.Model):
    user_id: int (PK, Auto)
    email: str (Unique, Not Null)
    username: str (Nullable)
    phone_number: str (Nullable)
    first_name: str (Nullable)
    last_name: str (Nullable)
    region: str (Nullable)          # 地區/縣市
    address: JSON (Nullable)
    role: UserRole (Not Null, Default: GENERAL_MEMBER)
    verified: bool (Not Null, Default: False)
    primary_shelter_id: int (FK -> Shelter, Nullable)
    profile_photo_url: str (Nullable)
    password_hash: str (Not Null)
    last_login_at: datetime (Nullable)
    failed_login_attempts: int (Default: 0)
    locked_until: datetime (Nullable)
    created_at: datetime (Not Null)
    updated_at: datetime (Not Null)
    deleted_at: datetime (Nullable)    # 軟刪除
```

關聯關係: - primary_shelter: Many-to-One → Shelter - animals: One-to-Many → Animal (owner_id) - applications: One-to-Many → Application (applicant_id) - notifications: One-to-Many → Notification (recipient_id)

JSON 輸出格式:


```
{
  "user_id": 1,
  "email": "user@example.com",
  "username": "example_user",
  "phone_number": "0912345678",
  "first_name": "John",
  "last_name": "Doe",
  "region": "台北市",
  "address": {
    "city": "台北市",
    "district": "中正區",
    "street": "重慶南路一段"
  },
  "role": "GENERAL_MEMBER",
  "verified": true,
  "primary_shelter_id": null,
  "profile_photo_url": "https://...",
  "created_at": "2025-12-01T10:00:00+08:00",
  "updated_at": "2025-12-25T12:00:00+08:00"
}
```

2.2 Animal (動物)

檔案位置: backend/app/models/animal.py

資料結構:

```python
class Species(Enum):
    CAT = 'CAT'
    DOG = 'DOG'

class Sex(Enum):
    MALE = 'MALE'
    FEMALE = 'FEMALE'
    UNKNOWN = 'UNKNOWN'

class AnimalStatus(Enum):
    DRAFT = 'DRAFT'             # 草稿
    SUBMITTED = 'SUBMITTED'     # 已提交審核
    PUBLISHED = 'PUBLISHED'     # 已發布
    ADOPTED = 'ADOPTED'         # 已被領養
```

```
    RETIRED = 'RETIRED'          # 已下架

class Animal(db.Model):
    animal_id: int (PK, Auto)
    name: str (Nullable)
    species: Species (Nullable)
    breed: str (Nullable)        # 品種
    color: str (Nullable)        # 毛色
    sex: Sex (Nullable)
    dob: date (Nullable)         # 出生日期
    description: text (Nullable)
    status: AnimalStatus (Not Null, Default: DRAFT)
    shelter_id: int (FK -> Shelter, Nullable)
    owner_id: int (FK -> User, Nullable)
    medical_summary: text (Nullable)
    rejection_reason: text (Nullable)
    rejected_at: datetime (Nullable)
    rejected_by: int (FK -> User, Nullable)
    created_by: int (FK -> User, Not Null)
    created_at: datetime (Not Null)
    updated_at: datetime (Not Null)
    deleted_at: datetime (Nullable)
```

關聯關係: - shelter: Many-to-One → Shelter - owner: Many-to-One → User - creator: Many-to-One → User - images: One-to-Many → AnimalImage - applications: One-to-Many → Application - medical_records: One-to-Many → MedicalRecord

JSON 輸出格式 (完整版，include_relations=True):


```
{
  "animal_id": 101,
  "name": "小白",
  "species": "DOG",
  "breed": "混種犬",
  "color": "白色",
  "sex": "MALE",
  "dob": "2023-05-15",
  "age": 1,
  "description": "活潑親人的狗狗",
  "status": "PUBLISHED",
  "shelter_id": null,
  "owner_id": 5,
  "medical_summary": "已絕育、已施打疫苗",
  "rejection_reason": null,
  "rejected_at": null,
  "rejected_by": null,
  "created_by": 5,
  "created_at": "2025-12-01T14:30:00+08:00",
  "updated_at": "2025-12-20T09:15:00+08:00",
```

```
  "images": [
    {
      "animal_image_id": 201,
      "storage_key": "uploads/5/abc123.jpg",
      "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
      "mime_type": "image/jpeg",
      "width": 1920,
      "height": 1080,
      "order": 0
    }
  ],
  "has_pending_application": false
}
```

2.3 Application (領養申請)

檔案位置: backend/app/models/application.py

資料結構:

```
class ApplicationType(Enum):
    ADOPTION = 'ADOPTION'
    REHOME = 'REHOME'

class ApplicationStatus(Enum):
    PENDING = 'PENDING'
    UNDER_REVIEW = 'UNDER_REVIEW'
    APPROVED = 'APPROVED'
    REJECTED = 'REJECTED'
    WITHDRAWN = 'WITHDRAWN'

class Application(db.Model):
    application_id: int (PK, Auto)
    applicant_id: int (FK -> User, Not Null)
    animal_id: int (FK -> Animal, Not Null)
    type: ApplicationType (Not Null)
    status: ApplicationStatus (Not Null, Default: PENDING)
    submitted_at: datetime (Nullable)
    reviewed_at: datetime (Nullable)
    review_notes: text (Nullable)
    assignee_id: int (FK -> User, Nullable)
    version: int (Not Null, Default: 1)  # Optimistic locking
    idempotency_key: str (Unique, Nullable)
    attachments: JSON (Nullable)
    # 申請人詳細資料
    contact_phone: str (Nullable)
    contact_address: str (Nullable)
    occupation: str (Nullable)
```

housing_type: str (Nullable)
has_experience: bool (Nullable)
reason: text (Nullable)
notes: text (Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable)

JSON 輸出格式:


```
{
 "application_id": 301,
 "applicant_id": 10,
 "animal_id": 101,
 "type": "ADOPTION",
 "status": "PENDING",
 "submitted_at": "2025-12-24T16:30:00+08:00",
 "reviewed_at": null,
 "review_notes": null,
 "assignee_id": null,
 "version": 1,
 "attachments": null,
 "contact_phone": "0912345678",
 "contact_address": "台北市中正區",
 "occupation": "工程師",
 "housing_type": "公寓",
 "has_experience": true,
 "reason": "想給狗狗一個溫暖的家",
 "notes": "平日晚上 7 點後可聯絡",
 "created_at": "2025-12-24T16:30:00+08:00",
 "updated_at": "2025-12-24T16:30:00+08:00"
}
```

2.4 Shelter (收容所)

檔案位置: backend/app/models/shelter.py

資料結構:

```
class Shelter(db.Model):
    shelter_id: int (PK, Auto)
    name: str (Not Null)
    slug: str (Unique, Nullable)
    contact_email: str (Not Null)
    contact_phone: str (Not Null)
    address: JSON (Not Null)          # {street, city, county, postal_code}
    region: str (Nullable)            # 地區/縣市
    verified: bool (Not Null, Default: False)
```

primary_account_user_id: int (FK -> User, Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable)

關聯關係: - primary_account: Many-to-One → User - primary_users: One-to-Many → User (primary_shelter_id) - animals: One-to-Many → Animal

JSON 輸出格式:

```
{
 "shelter_id": 50,
 "name": "台北市動物之家",
 "slug": "taipei-animal-shelter",
 "contact_email": "contact@taipei-shelter.gov.tw",
 "contact_phone": "02-12345678",
 "address": {
  "street": "內湖路一段",
  "city": "台北市",
  "county": "內湖區",
  "postal_code": "114"
 },
 "region": "台北市",
 "verified": true,
 "primary_account_user_id": 8,
 "created_at": "2025-01-15T09:00:00+08:00",
 "updated_at": "2025-12-20T14:30:00+08:00"
}
```

2.5 MedicalRecord (醫療紀錄)

檔案位置: backend/app/models/medical_record.py

資料結構:

```
class RecordType(Enum):
    TREATMENT = 'TREATMENT'  # 治療
    CHECKUP = 'CHECKUP'      # 健康檢查
    VACCINE = 'VACCINE'      # 疫苗接種
    SURGERY = 'SURGERY'      # 手術
    OTHER = 'OTHER'          # 其他

class MedicalRecord(db.Model):
    medical_record_id: int (PK, Auto)
    animal_id: int (FK -> Animal, Not Null)
    record_type: RecordType (Nullable)
    date: date (Nullable)            # 醫療日期
```

```
provider: str (Nullable)        # 醫療機構/獸醫
details: text (Nullable)        # 詳細說明
attachments: JSON (Nullable)    # 附件清單
verified: bool (Not Null, Default: False)
verified_by: int (FK -> User, Nullable)
created_by: int (FK -> User, Nullable)
created_at: datetime (Not Null)
updated_at: datetime (Not Null)
deleted_at: datetime (Nullable)
```

關聯關係: - animal: Many-to-One → Animal - verifier: Many-to-One → User - creator: Many-to-One → User

JSON 輸出格式:


```
{
 "medical_record_id": 401,
 "animal_id": 101,
 "record_type": "VACCINE",
 "date": "2025-12-01",
 "provider": "台北動物醫院",
 "details": "狂犬病疫苗接種(第二劑)",
 "attachments": [
  {
   "attachment_id": 501,
   "url": "http://localhost:9000/petadopt/uploads/medical/...",
   "filename": "vaccine_certificate.pdf"
  }
 ],
 "verified": true,
 "verified_by": 1,
 "created_by": 5,
 "created_at": "2025-12-01T15:30:00+08:00",
 "updated_at": "2025-12-02T10:00:00+08:00"
}
```

2.6 Notification (通知)

檔案位置: backend/app/models/others.py

資料結構:

```
class Notification(db.Model):
    notification_id: int (PK, Auto)
    recipient_id: int (FK -> User, Not Null)
    actor_id: int (FK -> User, Nullable)
    type: str (Not Null)            # 通知類型
    payload: JSON (Nullable)        # 通知內容
```

read: bool (Not Null, Default: False)
created_at: datetime (Not Null)
read_at: datetime (Nullable)

通知類型: - application_submitted: 申請已提交 - application_approved: 申請已核准 - application_rejected: 申請已拒絕 - animal_published: 動物已發布 - animal_adopted: 動物已領養

JSON 輸出格式:

```
{
 "notification_id": 601,
 "recipient_id": 10,
 "actor_id": 5,
 "type": "application_approved",
 "payload": {
  "application_id": 301,
  "animal_id": 101,
  "animal_name": "小白",
  "message": "您的領養申請已通過審核"
 },
 "read": false,
 "created_at": "2025-12-25T15:00:00+08:00",
 "read_at": null
}
```

2.7 Job (背景任務)

檔案位置: backend/app/models/others.py

資料結構:

```
class JobType(Enum):
    IMPORT_ANIMALS = 'import_animals'
    EXPORT_USER_DATA = 'export_user_data'
    BATCH_NOTIFICATION = 'batch_notification'
    GENERATE_REPORT = 'generate_report'

class JobStatus(Enum):
    PENDING = 'PENDING'
    RUNNING = 'RUNNING'
    SUCCEEDED = 'SUCCEEDED'
    FAILED = 'FAILED'

class Job(db.Model):
    job_id: int (PK, Auto)
    type: str (Not Null)
    status: JobStatus (Not Null, Default: PENDING)
```

```
    payload: JSON (Nullable)        # 任務參數
    result_summary: JSON (Nullable)   # 執行結果摘要
    created_by: int (FK -> User, Nullable)
    created_at: datetime (Not Null)
    started_at: datetime (Nullable)
    finished_at: datetime (Nullable)
    attempts: int (Not Null, Default: 0)
```

JSON 輸出格式:


```
{
  "job_id": 701,
  "type": "import_animals",
  "status": "SUCCEEDED",
  "payload": {
    "file_url": "uploads/batch/animals_20251225.csv",
    "shelter_id": 50
  },
  "result_summary": {
    "total": 100,
    "success": 95,
    "failed": 5,
    "errors": ["第 10 行: 缺少必填欄位 'name'"]
  },
  "created_by": 8,
  "created_at": "2025-12-25T10:00:00+08:00",
  "started_at": "2025-12-25T10:00:05+08:00",
  "finished_at": "2025-12-25T10:02:30+08:00",
  "attempts": 1
}
```

2.8 Attachment (附件)

檔案位置: backend/app/models/others.py

資料結構:

```
class Attachment(db.Model):
    attachment_id: int (PK, Auto)
    owner_type: str (Not Null)       # 擁有者類型 (e.g., 'medical_record')
    owner_id: int (Not Null)         # 擁有者 ID
    storage_key: str (Not Null)      # MinIO 儲存路徑
    url: str (Not Null)              # 完整 URL
    filename: str (Nullable)         # 原始檔名
    mime_type: str (Nullable)        # MIME 類型
    size: int (Nullable)             # 檔案大小 (bytes)
    meta_data: JSON (Nullable)       # 額外資訊
```

created_by: int (FK -> User, Nullable)
created_at: datetime (Not Null)
deleted_at: datetime (Nullable)

JSON 輸出格式:


```
{
 "attachment_id": 501,
 "owner_type": "medical_record",
 "owner_id": 401,
 "storage_key": "uploads/medical/uuid-abc.pdf",
 "url": "http://localhost:9000/petadopt/uploads/medical/uuid-abc.pdf",
 "filename": "vaccine_certificate.pdf",
 "mime_type": "application/pdf",
 "size": 524288,
 "meta_data": {
  "original_name": "狂犬病疫苗證明.pdf",
  "upload_source": "web"
 },
 "created_by": 5,
 "created_at": "2025-12-01T15:25:00+08:00"
}
```

2.9 AuditLog (稽核日誌)

檔案位置: backend/app/models/others.py

資料結構:

```
class AuditLog(db.Model):
    audit_log_id: int (PK, Auto)
    actor_id: int (FK -> User, Nullable)
    action: str (Not Null)          # 操作類型
    resource_type: str (Not Null)    # 資源類型
    resource_id: int (Nullable)      # 資源 ID
    changes: JSON (Nullable)         # 變更內容
    ip_address: str (Nullable)
    user_agent: str (Nullable)
    created_at: datetime (Not Null)
```

JSON 輸出格式:


```
{
 "audit_log_id": 801,
 "actor_id": 1,
 "action": "UPDATE",
 "resource_type": "Application",
 "resource_id": 301,
```

```
  "changes": {
    "status": {
      "old": "PENDING",
      "new": "APPROVED"
    },
    "review_notes": {
      "old": null,
      "new": "申請資料完整"
    }
  },
  "ip_address": "192.168.1.100",
  "user_agent": "Mozilla/5.0...",
  "created_at": "2025-12-25T15:00:00+08:00"
}
```

---

3. REST API 詳細規格

3.1 Authentication APIs

3.1.1 POST /auth/register

描述: 使用者註冊（產生待驗證記錄）

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "username": "example_user",
  "phone_number": "0912345678",
  "region": "台北市",
  "address": {
    "city": "台北市",
    "district": "中正區"
  }
}
```

Response (201 Created):

```
{
  "message": "驗證碼已發送至電子郵件",
  "pending_id": 12345,
  "masked_email": "use***@example.com",
  "expires_in": 900
}
```

錯誤回應 (409 Conflict):

```
{
 "message": "Email 已被註冊"
}
```

3.1.2 POST /auth/login

描述: 使用者登入

Request Body:


```
{
 "email": "user@example.com",
 "password": "SecurePass123!"
}
```


Response (200 OK):


```
{
 "message": "登入成功",
 "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "user": {
  "user_id": 1,
  "email": "user@example.com",
  "username": "example_user",
  "role": "GENERAL_MEMBER",
  "verified": true
 }
}
```


錯誤回應 (401 Unauthorized):


```
{
 "message": "Email 或密碼錯誤"
}
```

3.2 Animals APIs

3.2.1 GET /animals

描述: 取得動物列表（支援搜尋與篩選）

Query Parameters: - species: 物種 (CAT, DOG) - sex: 性別 (MALE, FEMALE, UNKNOWN) - status: 狀態 (預設: PUBLISHED) - shelter_id: 收容所 ID - source_type: 來源類型 (shelter, personal) - region: 地區/縣市 - min_age: 最小年齡（月數） - max_age: 最大年齡（月數） - q: 搜尋關鍵字 - page: 頁碼 (預設: 1) - per_page: 每頁筆數 (預設: 20, 最大: 100)

Request Example:

GET /animals?species=DOG&region=台北市&page=1&per_page=12

Response (200 OK):


```
{
  "animals": [
    {
      "animal_id": 101,
      "name": "小白",
      "species": "DOG",
      "breed": "混種犬",
      "sex": "MALE",
      "age": 1,
      "status": "PUBLISHED",
      "region": "台北市",
      "images": [
        {
          "animal_image_id": 201,
          "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
          "order": 0
        }
      ],
      "has_pending_application": false
    }
  ],
  "page": 1,
  "per_page": 12,
  "total": 45,
  "pages": 4
}
```

3.2.2 GET /animals/:id

描述: 取得單一動物詳細資訊

Path Parameters: - id: 動物 ID (integer)

Response (200 OK):


```
{
  "animal_id": 101,
  "name": "小白",
  "species": "DOG",
  "breed": "混種犬",
  "color": "白色",
  "sex": "MALE",
  "dob": "2023-05-15",
```

```
 "age": 1,
 "description": "活潑親人的狗狗，適合有經驗的飼主。",
 "status": "PUBLISHED",
 "shelter_id": null,
 "owner_id": 5,
 "medical_summary": "已絕育、已施打狂犬病疫苗",
 "created_by": 5,
 "created_at": "2025-12-01T14:30:00+08:00",
 "updated_at": "2025-12-20T09:15:00+08:00",
 "images": [
  {
   "animal_image_id": 201,
   "storage_key": "uploads/5/abc123.jpg",
   "url": "http://localhost:9000/petadopt/uploads/5/abc123.jpg",
   "mime_type": "image/jpeg",
   "width": 1920,
   "height": 1080,
   "order": 0
  }
 ],
 "owner": {
  "user_id": 5,
  "username": "john_doe",
  "region": "台北市"
 },
 "has_pending_application": false
}
```


錯誤回應 (404 Not Found):


```
{
 "message": "動物不存在"
}
```

3.2.3 POST /animals

描述: 建立動物資料（送養刊登） **需要認證**: Yes (JWT Bearer Token)

Request Headers:

Authorization: Bearer <access_token>
Content-Type: application/json

Request Body:


```
{
 "name": "小花",
 "species": "CAT",
 "breed": "米克斯",
```

```
 "color": "橘色",
 "sex": "FEMALE",
 "dob": "2024-03-10",
 "description": "溫柔親人的貓咪",
 "medical_summary": "已絕育",
 "status": "DRAFT"
}
```

Response (201 Created):

```
{
 "message": "動物資料建立成功",
 "animal": {
  "animal_id": 102,
  "name": "小花",
  "species": "CAT",
  "status": "DRAFT",
  "created_by": 5,
  "created_at": "2025-12-25T12:30:00+08:00"
 }
}
```

3.2.4 PATCH /animals/:id

描述: 更新動物資訊 **需要認證**: Yes **權限**: 僅限動物擁有者或管理員

Request Body (部分欄位):

```
{
 "description": "更新後的描述",
 "medical_summary": "已完成三合一疫苗"
}
```

Response (200 OK):

```
{
 "message": "動物資料更新成功",
 "animal": { /* 完整 animal 物件 */ }
}
```

3.3 Applications APIs

3.3.1 GET /applications

描述: 取得申請列表 **需要認證**: Yes

Query Parameters: - mode: 查詢模式 - all: 所有相關申請（管理員） - my: 我提交的申請（一般會員） - review: 待我審核的申請（收容所/管理員） - status: 申請狀態篩選 - animal_id: 特定動物的申請 - page, per_page: 分頁參數

Response (200 OK):


```
{
 "items": [
  {
   "application_id": 301,
   "applicant_id": 10,
   "animal_id": 101,
   "type": "ADOPTION",
   "status": "PENDING",
   "submitted_at": "2025-12-24T16:30:00+08:00",
   "contact_phone": "0912345678",
   "reason": "想給狗狗一個溫暖的家",
   "applicant": {
    "user_id": 10,
    "username": "adopter_01"
   },
   "animal": {
    "animal_id": 101,
    "name": "小白",
    "species": "DOG"
   }
  }
 ],
 "total": 15,
 "page": 1,
 "per_page": 20,
 "pages": 1
}
```

3.3.2 POST /applications

描述: 建立領養申請 **需要認證**: Yes (GENERAL_MEMBER only)

Request Headers:

Authorization: Bearer <access_token>
Idempotency-Key: <unique_key> // 可選，用於防止重複提交
Content-Type: application/json

Request Body:


```
{
 "animal_id": 101,
 "type": "ADOPTION",
```

```
 "contact_phone": "0912345678",
 "contact_address": "台北市中正區重慶南路",
 "occupation": "軟體工程師",
 "housing_type": "公寓",
 "has_experience": true,
 "reason": "我很喜歡狗狗，有充足時間陪伴",
 "notes": "平日晚上 7 點後可聯絡"
}
```


Response (201 Created):


```
{
 "message": "申請已提交",
 "application": {
  "application_id": 302,
  "applicant_id": 10,
  "animal_id": 101,
  "status": "PENDING",
  "submitted_at": "2025-12-25T14:00:00+08:00"
 }
}
```


Business Rules: 1. 申請者不能對自己刊登的動物提出申請 2. 同一動物只能提交一次申請（透過 idempotency_key 確保） 3. 僅限 GENERAL_MEMBER 角色

3.3.3 POST /applications/:id/review

描述: 審核申請（核准/拒絕） **需要認證**: Yes **權限**: 動物擁有者、收容所管理員、系統管理員

Request Body:


```
{
 "action": "approve",
 "review_notes": "申請資料完整，條件符合",
 "version": 1
}
```


Response (200 OK):


```
{
 "message": "申請已核准",
 "application": {
  "application_id": 301,
  "status": "APPROVED",
  "reviewed_at": "2025-12-25T15:00:00+08:00",
  "review_notes": "申請資料完整，條件符合"
```

```
  }
}
```

3.4 Uploads APIs

3.4.1 POST /uploads/direct

描述: 直接上傳檔案（後端代理至 MinIO） **需要認證**: Yes

Request: - Content-Type: multipart/form-data - Body: file (binary)

Response (200 OK):

```
{
  "message": "檔案上傳成功",
  "storage_key": "uploads/5/uuid-123.jpg",
  "url": "http://localhost:9000/petadopt/uploads/5/uuid-123.jpg",
  "size": 2048576,
  "mime_type": "image/jpeg"
}
```

3.4.2 POST /uploads/presign

描述: 取得預簽名 URL（前端直接上傳至 MinIO） **需要認證**: Yes

Request Body:

```
{
  "filename": "my-dog-photo.jpg",
  "mime_type": "image/jpeg",
  "size": 2048576
}
```

Response (200 OK):

```
{
  "presigned_url": "http://localhost:9000/petadopt/uploads/...",
  "storage_key": "uploads/5/uuid-456.jpg",
  "expires_in": 3600
}
```

3.5 Medical Records APIs

3.5.1 GET /medical-records/animals

描述: 取得當前使用者有權限管理醫療紀錄的動物列表 **需要認證**: Yes

Query Parameters: - name: 動物名稱搜尋 - species: 物種篩選 - breed: 品種篩選 - min_age, max_age: 年齡範圍 - adopted: 是否已領養

Response (200 OK):


```
{
 "animals": [
  {
   "animal_id": 101,
   "name": "小白",
   "species": "DOG",
   "breed": "混種犬",
   "age": 1,
   "status": "PUBLISHED"
  }
 ],
 "total": 5
}
```

3.5.2 POST /medical-records/animals/:animal_id/medical-records

描述: 為動物建立醫療紀錄 **需要認證**: Yes **權限**: 動物擁有者、收容所成員、管理員

Request Body:


```
{
 "record_type": "VACCINE",
 "date": "2025-12-25",
 "provider": "台北動物醫院",
 "details": "狂犬病疫苗接種",
 "attachments": [
  {
   "storage_key": "uploads/medical/uuid-123.pdf",
   "url": "http://...",
   "filename": "vaccine_cert.pdf",
   "mime_type": "application/pdf"
  }
 ]
}
```


Response (201 Created):


```
{
 "message": "醫療紀錄創建成功",
 "medical_record": {
  "medical_record_id": 401,
  "animal_id": 101,
  "record_type": "VACCINE",
  "date": "2025-12-25",
```

```
    "provider": "台北動物醫院",
    "details": "狂犬病疫苗接種",
    "verified": false
  }
}
```

3.5.3 GET /medical-records/animals/:animal_id/medical-records

描述: 取得動物的醫療紀錄列表

Response (200 OK):

```
{
  "medical_records": [
    {
      "medical_record_id": 401,
      "record_type": "VACCINE",
      "date": "2025-12-25",
      "provider": "台北動物醫院",
      "details": "狂犬病疫苗接種",
      "verified": true,
      "attachments": [...]
    }
  ],
  "total": 3
}
```

3.5.4 PATCH /medical-records/:id

描述: 更新醫療紀錄 **需要認證**: Yes **權限**: 建立者或管理員

Request Body (部分欄位):

```
{
  "details": "更新後的詳細說明",
  "verified": true
}
```

Response (200 OK):

```
{
  "message": "醫療紀錄更新成功",
  "medical_record": { /* 完整醫療紀錄 */ }
}
```

3.6 Shelters APIs

3.6.1 GET /shelters

描述: 取得收容所列表（公開端點）

Query Parameters: - page: 頁碼 - per_page: 每頁筆數 - search: 搜尋關鍵字 - verified: 是否只顯示已驗證的收容所 (true/false)

Response (200 OK):


```
{
 "shelters": [
  {
   "shelter_id": 50,
   "name": "台北市動物之家",
   "slug": "taipei-animal-shelter",
   "region": "台北市",
   "verified": true,
   "contact_phone": "02-12345678"
  }
 ],
 "page": 1,
 "per_page": 10,
 "total": 25,
 "pages": 3
}
```

3.6.2 POST /shelters

描述: 建立收容所 **需要認證**: Yes **權限**: SHELTER_MEMBER 或 ADMIN

Request Body:


```
{
 "name": "新竹市動物收容所",
 "contact_email": "contact@hsinchu-shelter.gov.tw",
 "contact_phone": "03-12345678",
 "address": {
  "street": "東大路一段",
  "city": "新竹市",
  "county": "東區",
  "postal_code": "300"
 },
 "region": "新竹市"
}
```


Response (201 Created):

```
{
 "message": "收容所創建成功",
 "shelter": {
  "shelter_id": 51,
  "name": "新竹市動物收容所",
  "verified": false,
  "primary_account_user_id": 8
 }
}
```

3.6.3 GET /shelters/:id

描述: 取得收容所詳細資訊（公開端點）

Response (200 OK):


```
{
 "shelter_id": 50,
 "name": "台北市動物之家",
 "slug": "taipei-animal-shelter",
 "contact_email": "contact@taipei-shelter.gov.tw",
 "contact_phone": "02-12345678",
 "address": {
  "street": "內湖路一段",
  "city": "台北市",
  "county": "內湖區",
  "postal_code": "114"
 },
 "region": "台北市",
 "verified": true,
 "primary_account_user_id": 8,
 "statistics": {
  "total_animals": 150,
  "published_animals": 120,
  "adopted_animals": 30
 }
}
```

3.6.4 POST /shelters/:id/animals/batch

描述: 批次匯入動物資料（CSV/JSON） **需要認證**: Yes **權限**: 收容所管理員

Request Body (JSON 格式):


```
{
 "animals": [
  {
```

```
    "name": "小黑",
    "species": "DOG",
    "breed": "拉布拉多",
    "sex": "MALE",
    "dob": "2023-01-15",
    "description": "友善活潑"
   },
   {
    "name": "小花",
    "species": "CAT",
    "breed": "米克斯",
    "sex": "FEMALE",
    "dob": "2024-03-10"
   }
  ]
}
```


Response (202 Accepted):


```
{
 "message": "批次匯入任務已建立",
 "job_id": 701,
 "status": "PENDING",
 "estimated_duration": 120
}
```


後續查詢任務狀態: GET /jobs/:job_id

3.7 Notifications APIs

3.7.1 GET /notifications

描述: 取得當前使用者的通知列表 **需要認證**: Yes

Query Parameters: - unread_only: 只顯示未讀通知 (true/false) - type: 通知類型篩選 - page, per_page: 分頁參數

Response (200 OK):


```
{
 "notifications": [
  {
   "notification_id": 601,
   "type": "application_approved",
   "payload": {
    "application_id": 301,
    "animal_name": "小白",
    "message": "您的領養申請已通過審核"
```

```
    },
    "read": false,
    "created_at": "2025-12-25T15:00:00+08:00",
    "actor": {
     "user_id": 5,
     "username": "john_doe"
    }
   }
 ],
 "total": 15,
 "unread_count": 3,
 "page": 1,
 "per_page": 20
}
```

3.7.2 POST /notifications/:id/read

描述: 標記通知為已讀 **需要認證**: Yes

Response (200 OK):


```
{
 "message": "通知已標記為已讀",
 "notification": {
  "notification_id": 601,
  "read": true,
  "read_at": "2025-12-25T16:00:00+08:00"
 }
}
```

3.7.3 POST /notifications/read-all

描述: 標記所有通知為已讀 **需要認證**: Yes

Response (200 OK):


```
{
 "message": "所有通知已標記為已讀",
 "updated_count": 5
}
```

3.7.4 GET /notifications/unread-count

描述: 取得未讀通知數量 **需要認證**: Yes

Response (200 OK):

```
{
  "unread_count": 3
}
```

3.8 Jobs APIs

3.8.1 GET /jobs/:id

描述: 取得任務狀態 **需要認證**: Yes

Response (200 OK):


```
{
  "job_id": 701,
  "type": "import_animals",
  "status": "RUNNING",
  "payload": {
    "shelter_id": 50,
    "file_url": "uploads/batch/animals.csv"
  },
  "result_summary": null,
  "created_at": "2025-12-25T10:00:00+08:00",
  "started_at": "2025-12-25T10:00:05+08:00",
  "finished_at": null,
  "attempts": 1,
  "progress": {
    "current": 50,
    "total": 100,
    "percentage": 50
  }
}
```

3.8.2 GET /jobs

描述: 取得任務列表 **需要認證**: Yes

Query Parameters: - status: 任務狀態篩選 - type: 任務類型篩選 - page, per_page: 分頁參數

Response (200 OK):


```
{
  "jobs": [
    {
      "job_id": 701,
      "type": "import_animals",
      "status": "SUCCEEDED",
      "created_at": "2025-12-25T10:00:00+08:00",
```

```
    "finished_at": "2025-12-25T10:02:30+08:00"
   }
 ],
 "total": 10,
 "page": 1,
 "per_page": 20
}
```

3.9 Users APIs

3.9.1 GET /users/:id

描述: 取得使用者公開資料

Response (200 OK):


```
{
 "user_id": 5,
 "username": "john_doe",
 "region": "台北市",
 "profile_photo_url": "http://...",
 "verified": true,
 "created_at": "2025-01-01T00:00:00+08:00"
}
```

3.9.2 PATCH /users/:id

描述: 更新使用者資料 **需要認證**: Yes **權限**: 本人或管理員

Request Body:


```
{
 "username": "new_username",
 "phone_number": "0987654321",
 "region": "新北市",
 "address": {
  "city": "新北市",
  "district": "板橋區"
 }
}
```


Response (200 OK):


```
{
 "message": "使用者資料更新成功",
 "user": { /* 完整使用者資料 */ }
}
```

3.10 Admin APIs

3.10.1 GET /admin/users

描述: 取得所有使用者列表（管理後台） **需要認證**: Yes **權限**: ADMIN only

Query Parameters: - role: 角色篩選 - verified: 驗證狀態篩選 - search: 搜尋 email/username - page, per_page: 分頁參數

Response (200 OK):

```
{
  "users": [
    {
      "user_id": 1,
      "email": "admin@example.com",
      "username": "admin",
      "role": "ADMIN",
      "verified": true,
      "last_login_at": "2025-12-25T09:00:00+08:00"
    }
  ],
  "total": 500,
  "page": 1,
  "per_page": 50
}
```

3.10.2 POST /admin/users/:id/update-role

描述: 更新使用者角色 **需要認證**: Yes **權限**: ADMIN only

Request Body:

```
{
  "role": "SHELTER_MEMBER"
}
```

Response (200 OK):

```
{
  "message": "使用者角色已更新",
  "user": {
    "user_id": 10,
    "role": "SHELTER_MEMBER"
  }
}
```

3.10.3 GET /admin/audit-logs

描述: 取得稽核日誌 **需要認證**: Yes **權限**: ADMIN only

Query Parameters: - actor_id: 操作者 ID - action: 操作類型 (CREATE, UPDATE, DELETE) - resource_type: 資源類型 - start_date, end_date: 日期範圍 - page, per_page: 分頁參數

Response (200 OK):


```
{
  "logs": [
    {
      "audit_log_id": 801,
      "actor_id": 1,
      "action": "UPDATE",
      "resource_type": "Application",
      "resource_id": 301,
      "changes": {
        "status": {"old": "PENDING", "new": "APPROVED"}
      },
      "ip_address": "192.168.1.100",
      "created_at": "2025-12-25T15:00:00+08:00"
    }
  ],
  "total": 1000,
  "page": 1,
  "per_page": 50
}
```


---

4. 前端資料結構 (TypeScript Interfaces)

4.1 Animals API Types

檔案位置: frontend/src/api/animals.ts

```typescript
export interface AnimalFilters {
  species?: 'CAT' | 'DOG'
  breed?: string
  sex?: 'MALE' | 'FEMALE' | 'UNKNOWN'
  status?: 'DRAFT' | 'SUBMITTED' | 'PUBLISHED' | 'ADOPTED' | 'RETIRED'
  shelter_id?: number
  owner_id?: number
  source_type?: 'shelter' | 'personal'
  region?: string
  min_age?: number
  max_age?: number
  featured?: boolean
```

```
  q?: string
  page?: number
  per_page?: number
}

export interface Animal {
  animal_id: number
  name?: string
  species?: 'CAT' | 'DOG'
  breed?: string
  color?: string
  sex?: 'MALE' | 'FEMALE' | 'UNKNOWN'
  dob?: string
  age?: number
  description?: string
  status: 'DRAFT' | 'SUBMITTED' | 'PUBLISHED' | 'ADOPTED' | 'RETIRED'
  shelter_id?: number
  owner_id?: number
  medical_summary?: string
  rejection_reason?: string
  rejected_at?: string
  rejected_by?: number
  created_by: number
  created_at: string
  updated_at: string
  deleted_at?: string
  images?: Array<{
    animal_image_id: number
    storage_key: string
    url: string
    mime_type?: string
    width?: number
    height?: number
    order: number
  }>
  featured?: boolean
  has_pending_application?: boolean
}

export interface AnimalsResponse {
  animals: Animal[]
  page: number
  per_page: number
  total: number
  pages: number
}
```

4.2 Applications API Types

檔案位置: frontend/src/api/applications.ts

```typescript
export interface Application {
  application_id: number
  applicant_id: number
  animal_id: number
  type: 'ADOPTION' | 'REHOME'
  status: 'PENDING' | 'UNDER_REVIEW' | 'APPROVED' | 'REJECTED' | 'WITHDRAWN'
  submitted_at?: string
  reviewed_at?: string
  review_notes?: string
  assignee_id?: number
  version: number
  attachments?: any
  contact_phone?: string
  contact_address?: string
  occupation?: string
  housing_type?: string
  has_experience?: boolean
  reason?: string
  notes?: string
  created_at: string
  updated_at: string
  applicant?: any
  animal?: any
  assignee?: any
}

export interface CreateApplicationData {
  animal_id: number
  type?: 'ADOPTION' | 'REHOME'
  attachments?: any
  contact_phone?: string
  contact_address?: string
  occupation?: string
  housing_type?: string
  has_experience?: boolean
  reason?: string
  notes?: string
}

export interface ReviewApplicationData {
  action: 'approve' | 'reject'
  review_notes?: string
  version?: number
}
```

4.3 Medical Records API Types

檔案位置: frontend/src/api/medicalRecords.ts

```
export interface MedicalRecord {
  medical_record_id: number
  animal_id: number
  record_type?: 'TREATMENT' | 'CHECKUP' | 'VACCINE' | 'SURGERY' | 'OTHER'
  date?: string
  provider?: string
  details?: string
  attachments?: Array<{
    attachment_id: number
    url: string
    filename?: string
    mime_type?: string
  }>
  verified: boolean
  verified_by?: number
  created_by?: number
  created_at: string
  updated_at: string
}

export interface CreateMedicalRecordData {
  animal_id: number
  record_type?: 'TREATMENT' | 'CHECKUP' | 'VACCINE' | 'SURGERY' | 'OTHER'
  date?: string
  provider?: string
  details?: string
  attachments?: Array<{
    storage_key: string
    url: string
    filename?: string
    mime_type?: string
  }>
}
```

4.4 Shelters API Types

檔案位置: frontend/src/api/shelters.ts

```
export interface Shelter {
  shelter_id: number
  name: string
  slug?: string
  contact_email: string
  contact_phone: string
```

```
  address: {
    street: string
    city: string
    county: string
    postal_code: string
  }
  region?: string
  verified: boolean
  primary_account_user_id?: number
  created_at: string
  updated_at: string
}

export interface ShelterFilters {
  page?: number
  per_page?: number
  search?: string
  verified?: boolean
}

export interface CreateShelterData {
  name: string
  contact_email: string
  contact_phone: string
  address: {
    street: string
    city: string
    county: string
    postal_code: string
  }
  region?: string
}
```

4.5 Notifications API Types

檔案位置: frontend/src/api/notifications.ts

```
export interface Notification {
  notification_id: number
  recipient_id: number
  actor_id?: number
  type: string
  payload: {
    [key: string]: any
  }
  read: boolean
  created_at: string
  read_at?: string
```

```
  actor?: {
    user_id: number
    username?: string
  }
}

export interface NotificationsResponse {
  notifications: Notification[]
  total: number
  unread_count: number
  page: number
  per_page: number
}

export interface NotificationFilters {
  unread_only?: boolean
  type?: string
  page?: number
  per_page?: number
}
```

4.6 Jobs API Types

檔案位置: frontend/src/api/jobs.ts

```
export interface Job {
  job_id: number
  type: string
  status: 'PENDING' | 'RUNNING' | 'SUCCEEDED' | 'FAILED'
  payload?: {
    [key: string]: any
  }
  result_summary?: {
    total?: number
    success?: number
    failed?: number
    errors?: string[]
  }
  created_by?: number
  created_at: string
  started_at?: string
  finished_at?: string
  attempts: number
  progress?: {
    current: number
    total: number
    percentage: number
  }
```

```
}

export interface JobsResponse {
  jobs: Job[]
  total: number
  page: number
  per_page: number
}
```

4.7 Uploads API Types

檔案位置: frontend/src/api/uploads.ts

```
export interface UploadDirectResponse {
  message: string
  storage_key: string
  url: string
  size: number
  mime_type: string
}

export interface PresignResponse {
  presigned_url: string
  storage_key: string
  expires_in: number
}

export interface PresignRequest {
  filename: string
  mime_type: string
  size: number
}
```

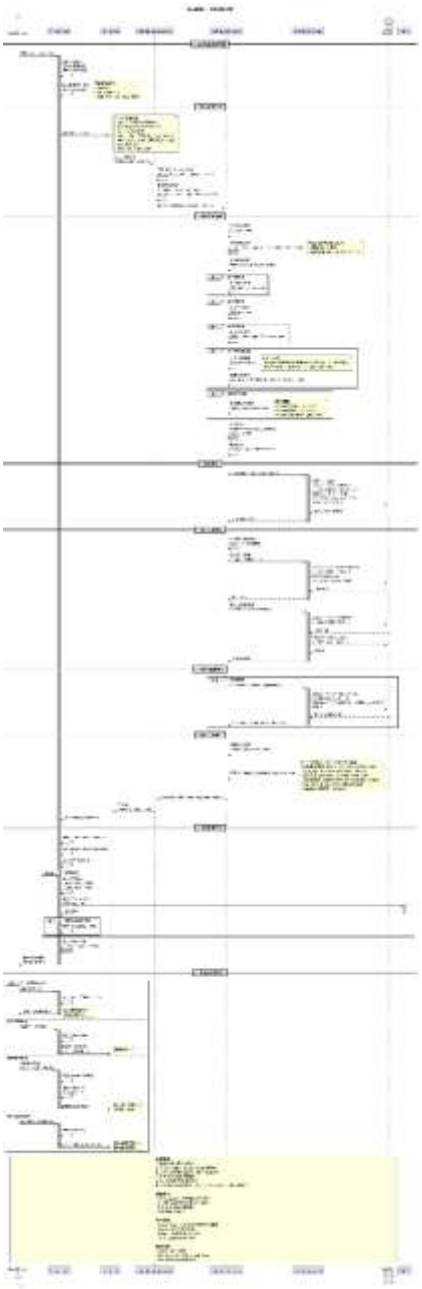
---

5. 循序圖索引

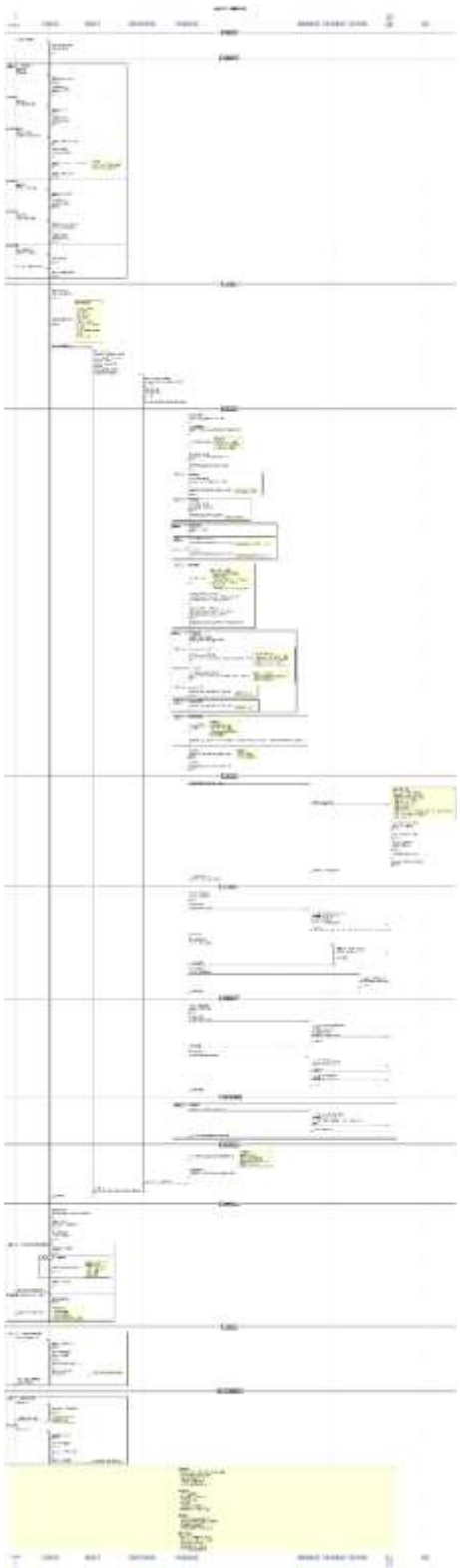
以下為各 Use Case 的循序圖(因圖片若太清楚會導致 pdf 加載緩慢，若想查看較清楚的循序圖，可參見 sd_img 資料夾)：

5.1 動物瀏覽與搜尋

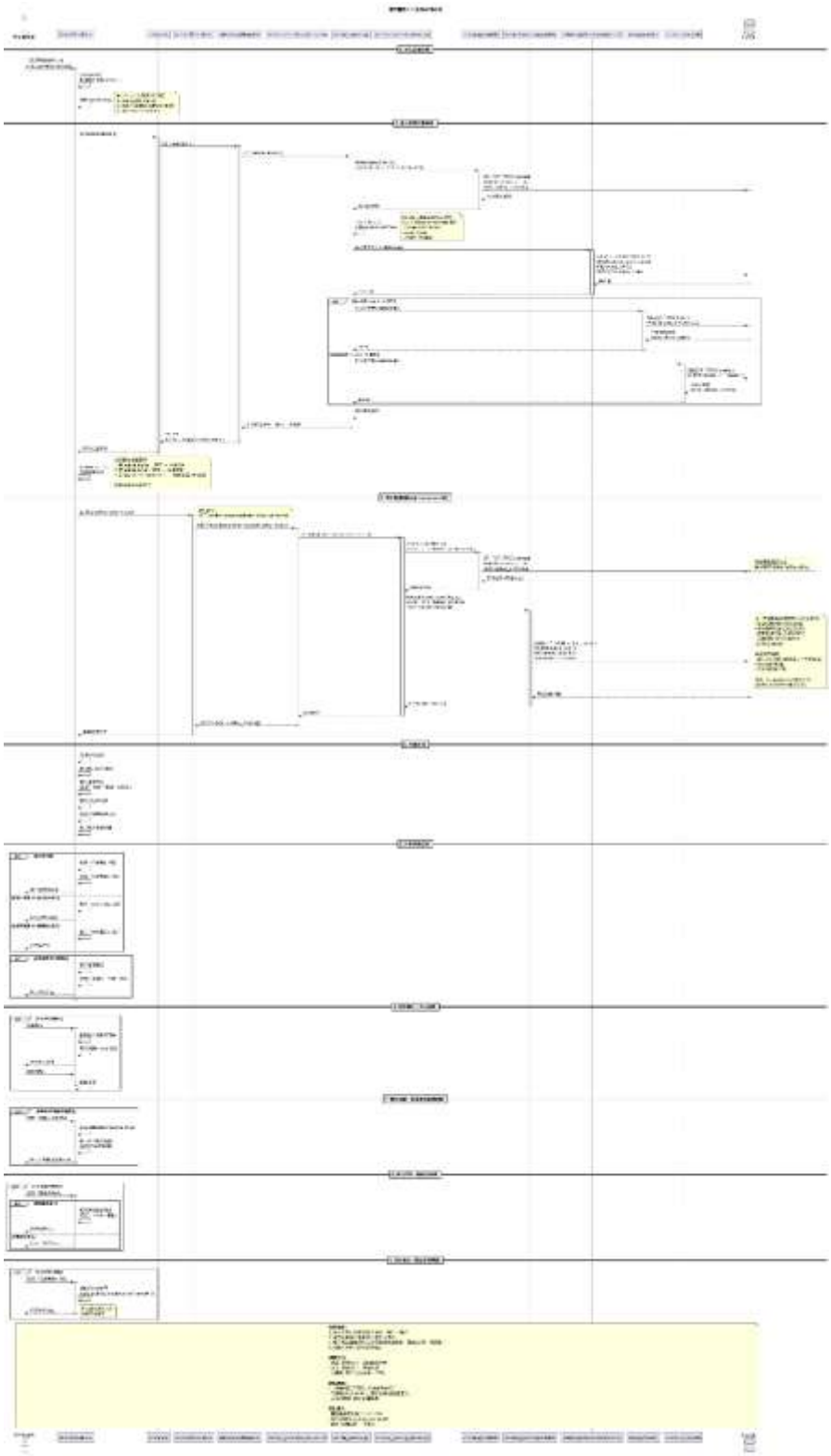
1_1: 動物列表瀏覽



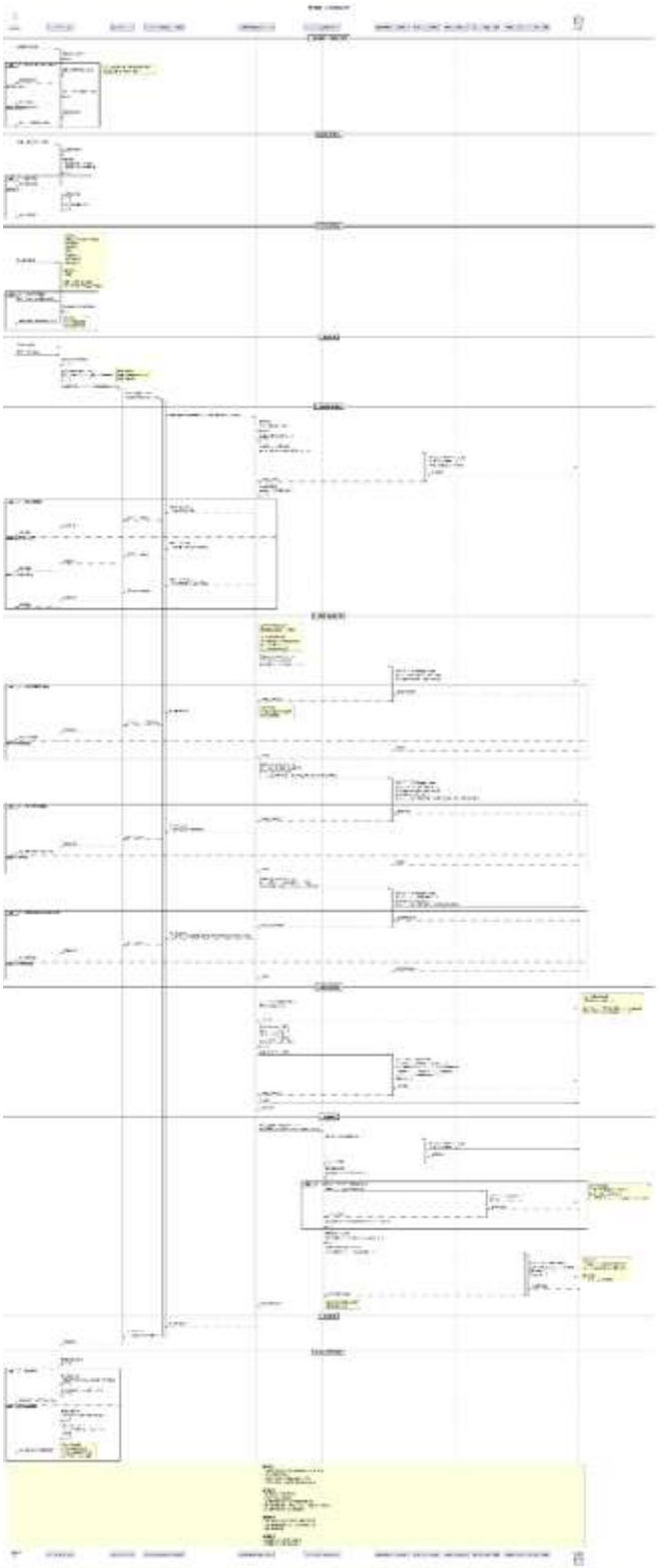
1_2: 動物搜尋篩選



1_3: 動物詳情檢視

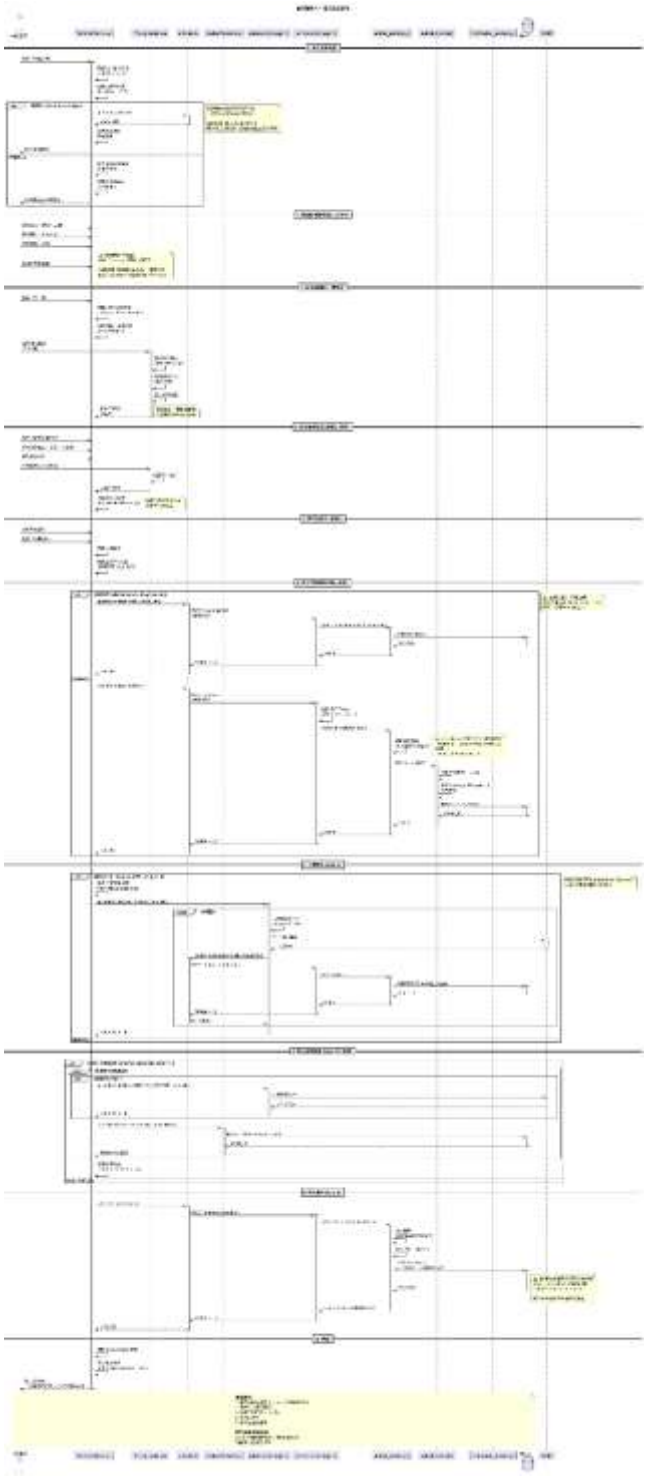


1_4: 領養申請提交

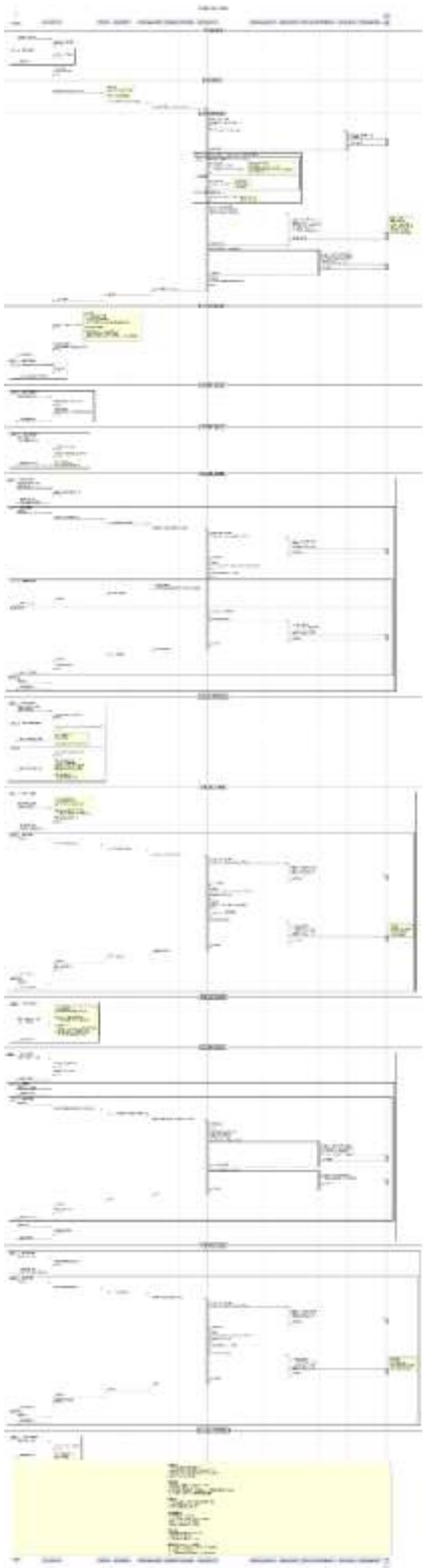


5.2 個人送養流程

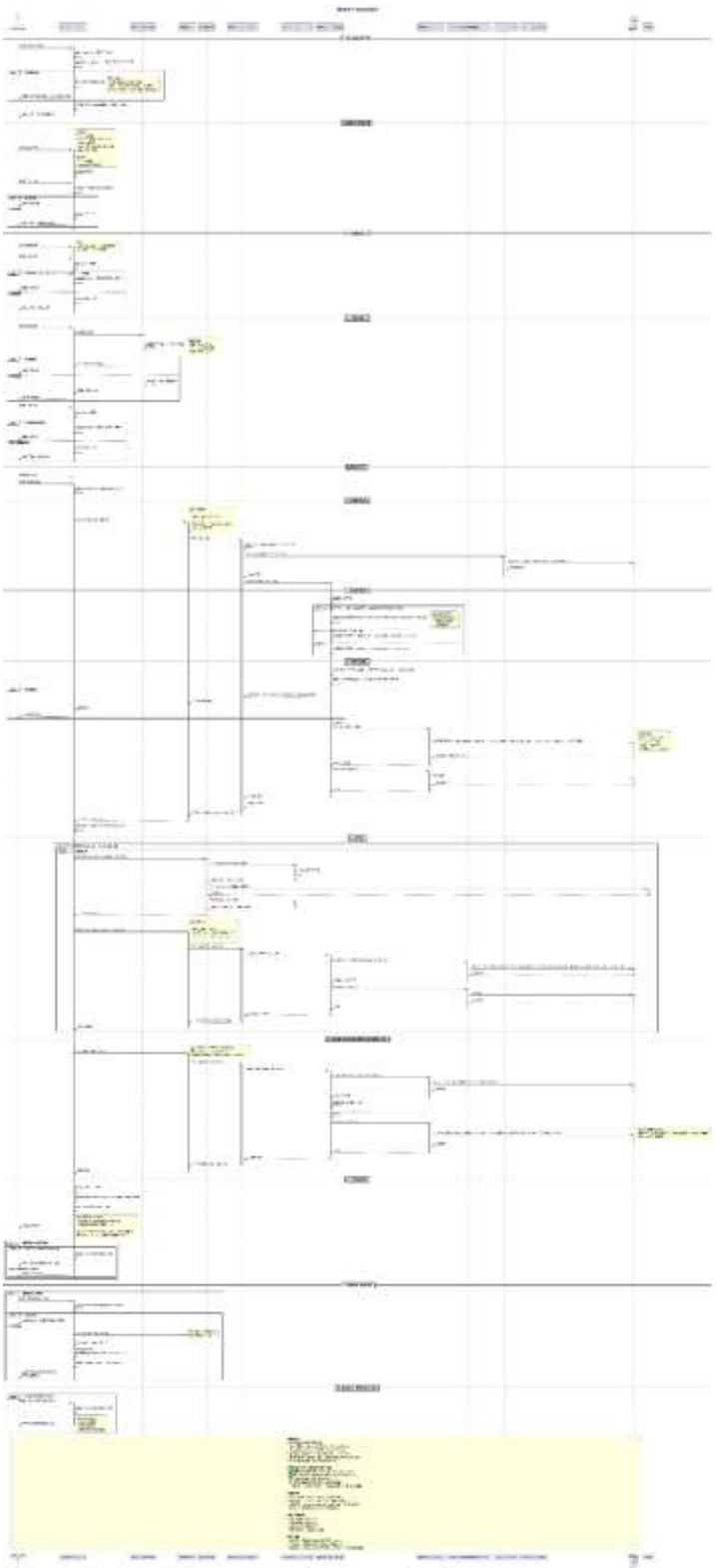
2_1: 個人送養發佈



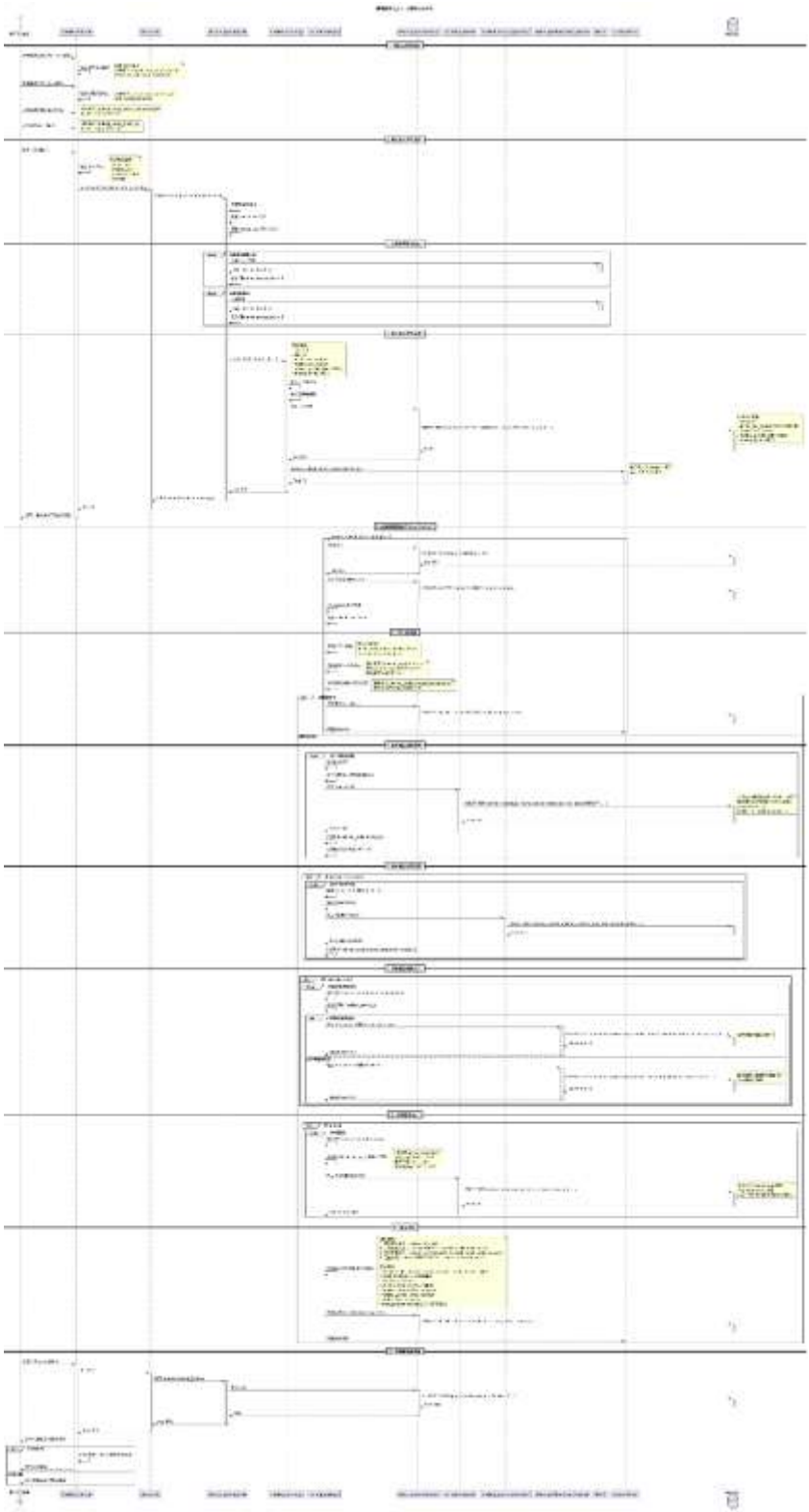
2_2: 個人送養管理



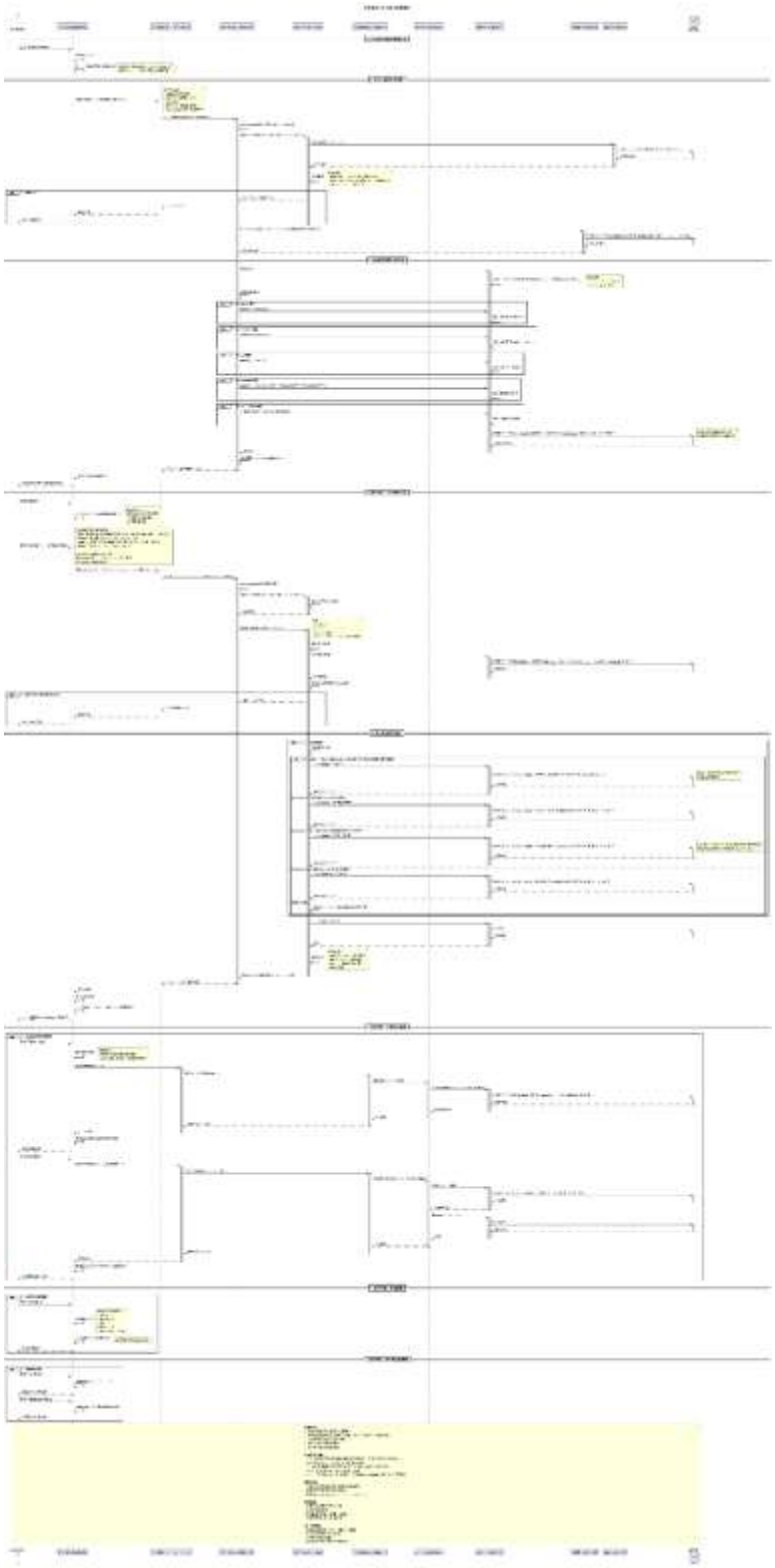
2_3: 收容所送養發佈



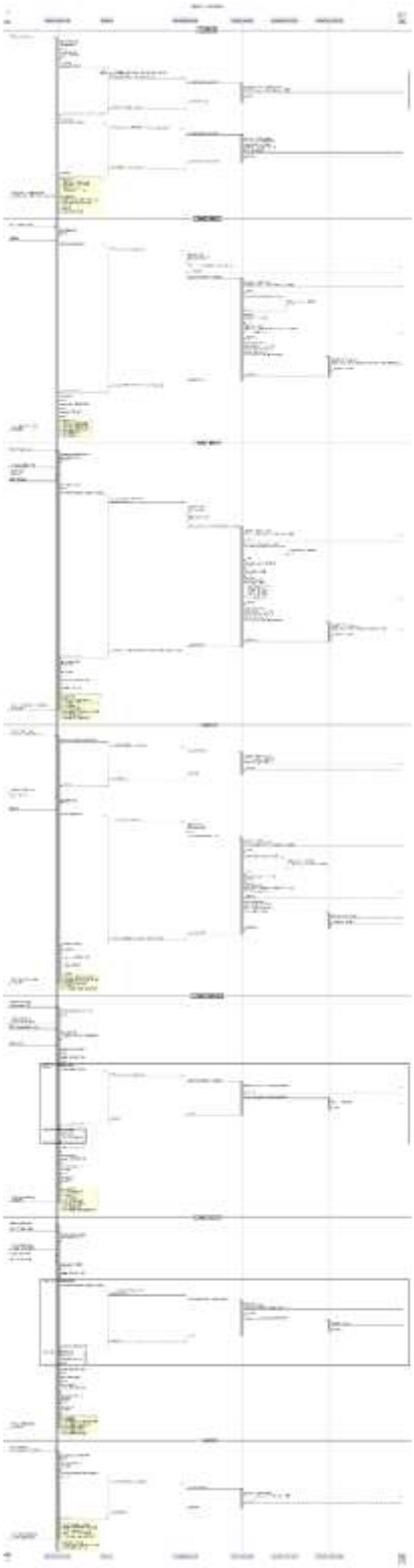
2_3_1: 送養提交審核（批次匯入）



2_4: 收容所送養管理

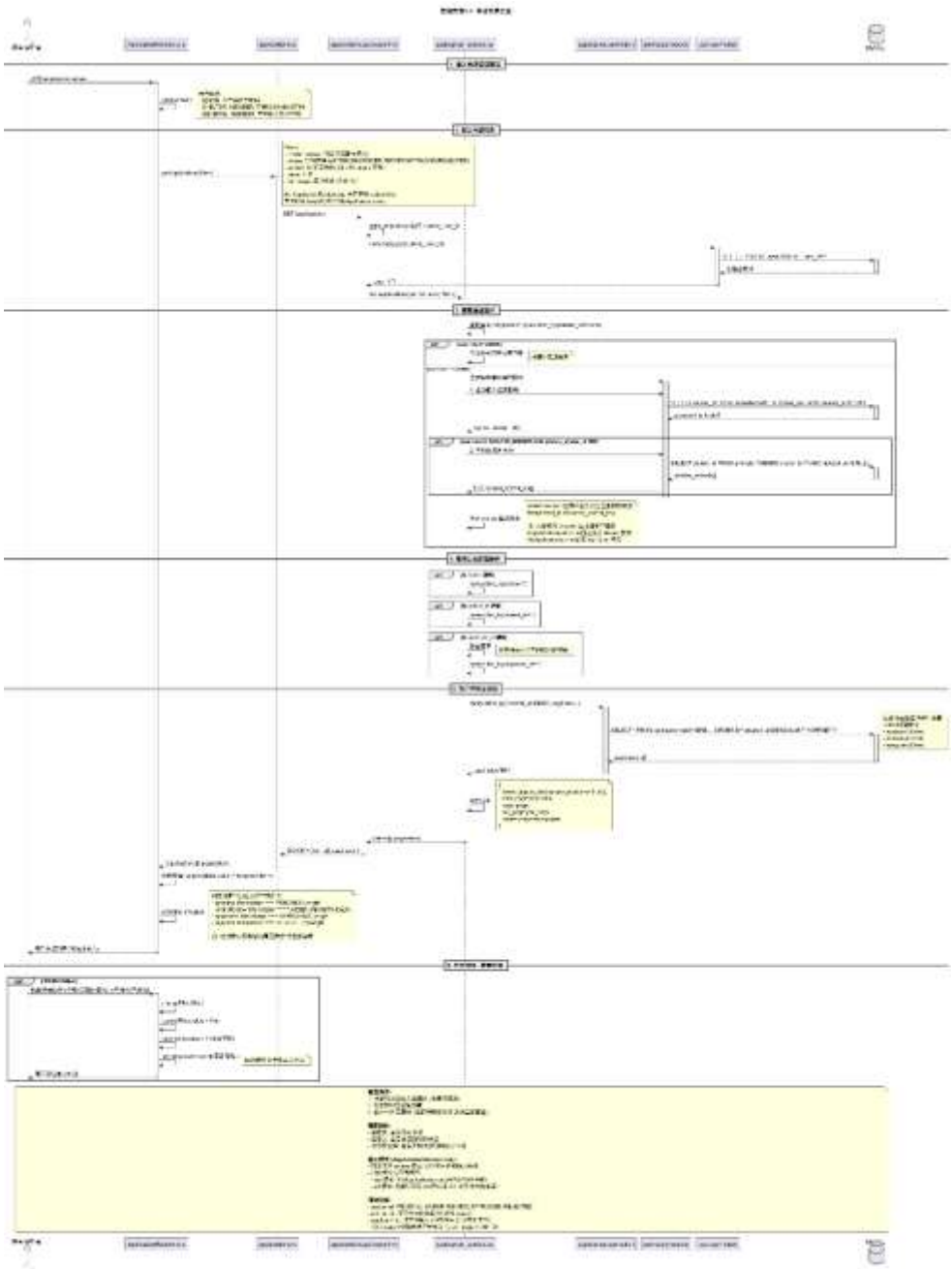


2_5: 送養刊登管理

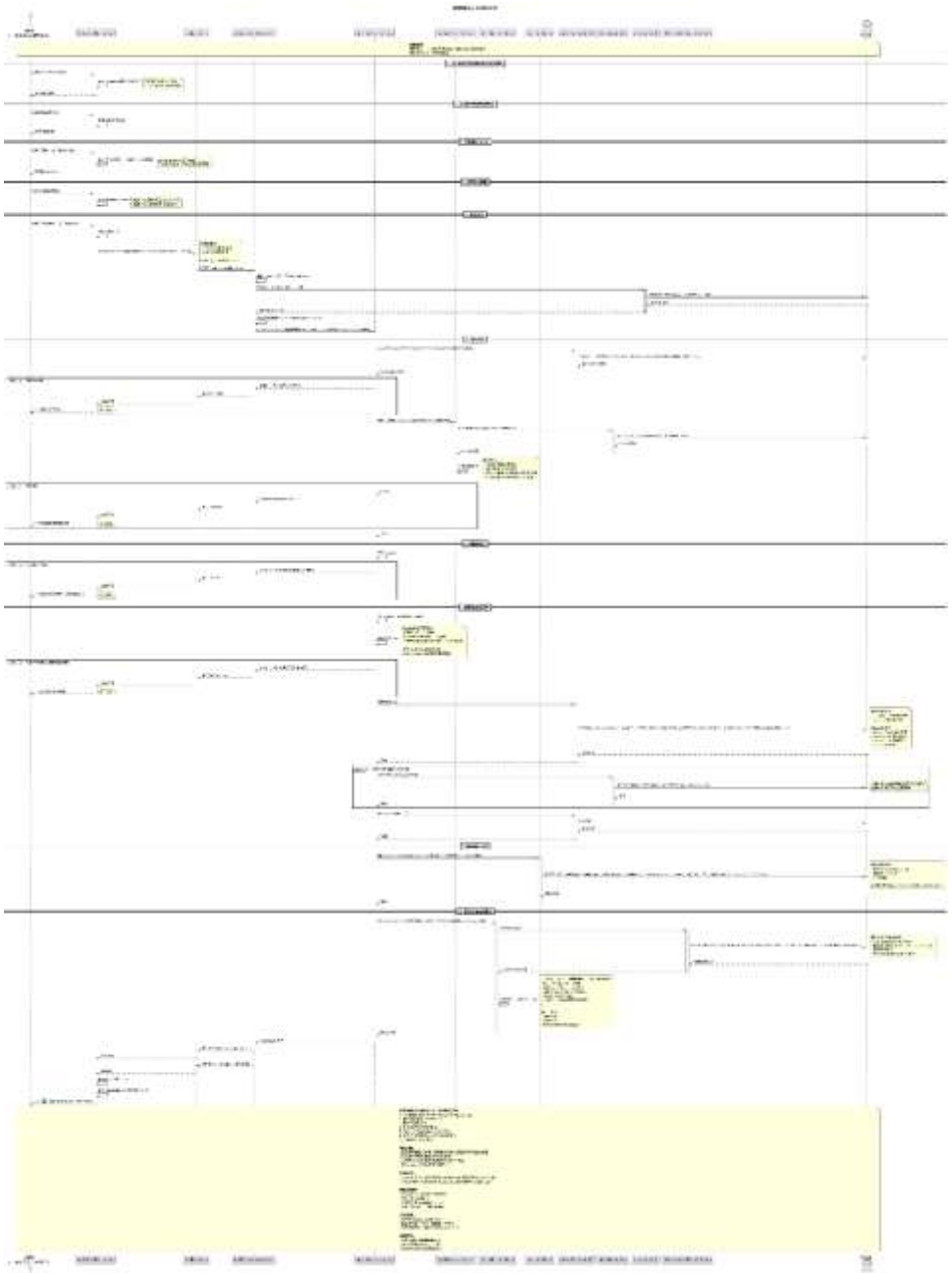


5.3 申請審核

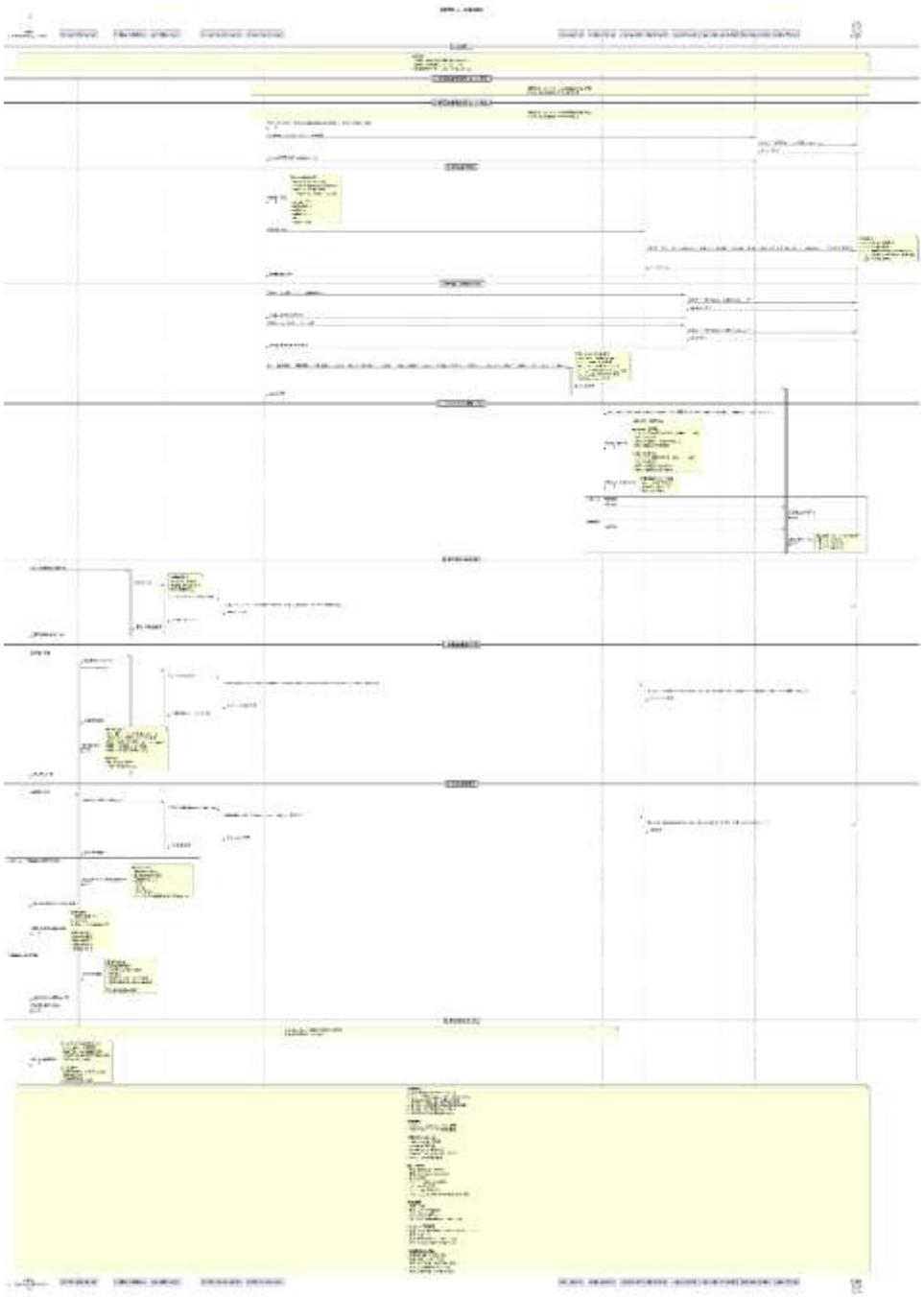
3_1: 申請列表瀏覽



3_2: 申請審核作業

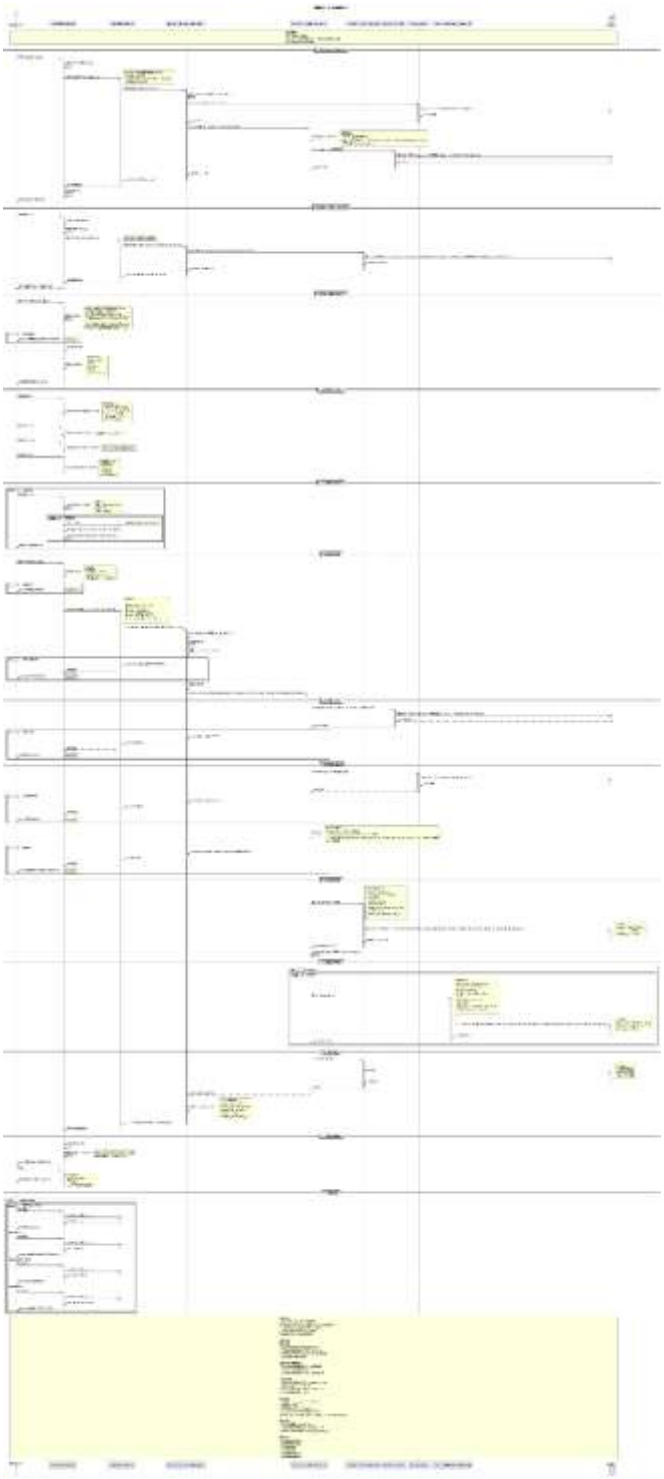


3_3: 申請審核通知

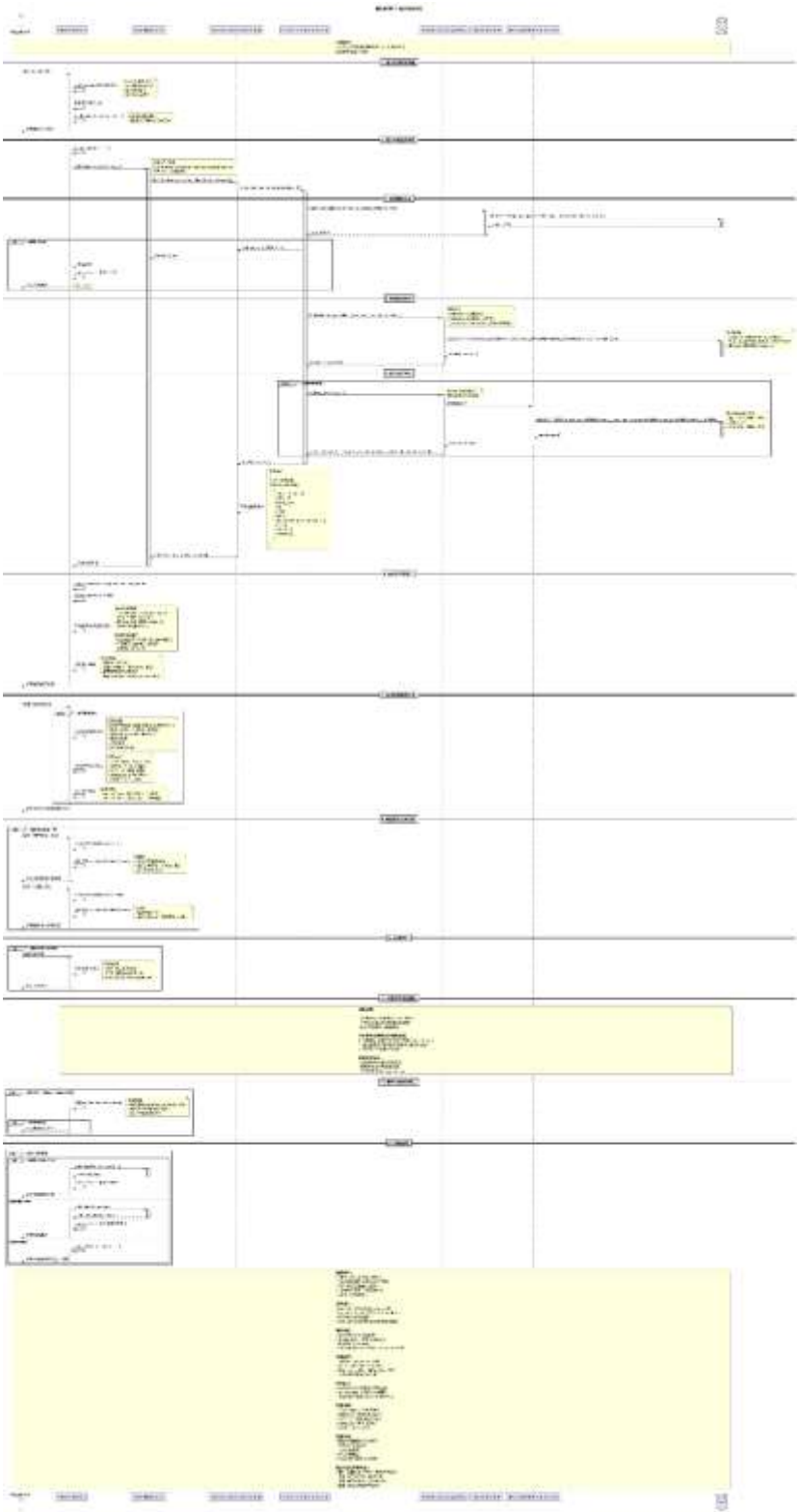


5.4 醫療紀錄

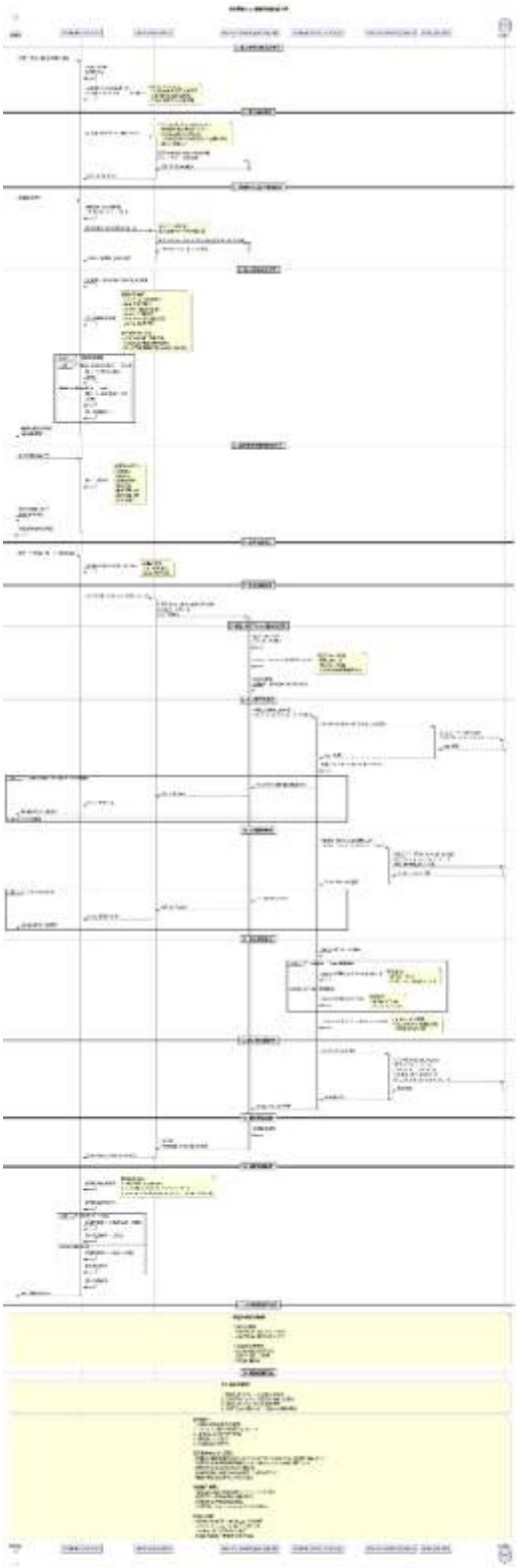
4_1: 醫療紀錄新增



4_2: 醫療紀錄檢視

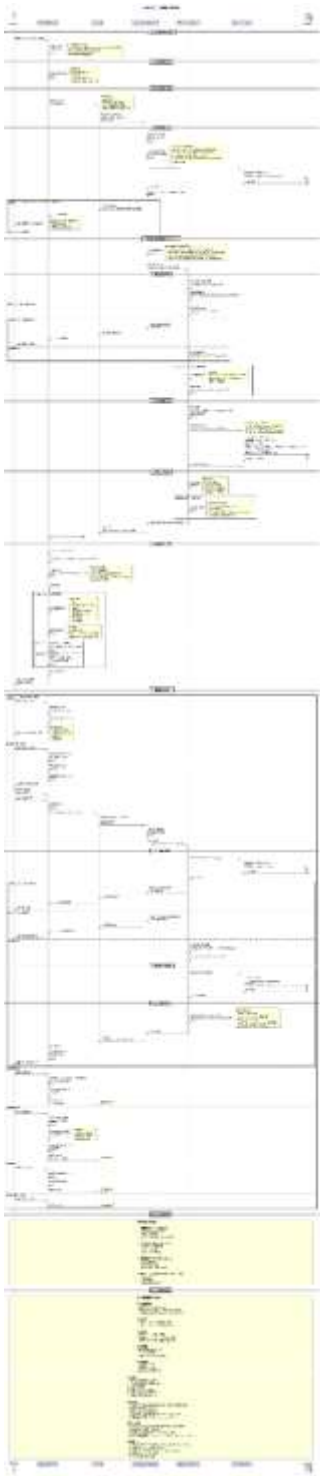


4_3: 醫療紀錄驗證

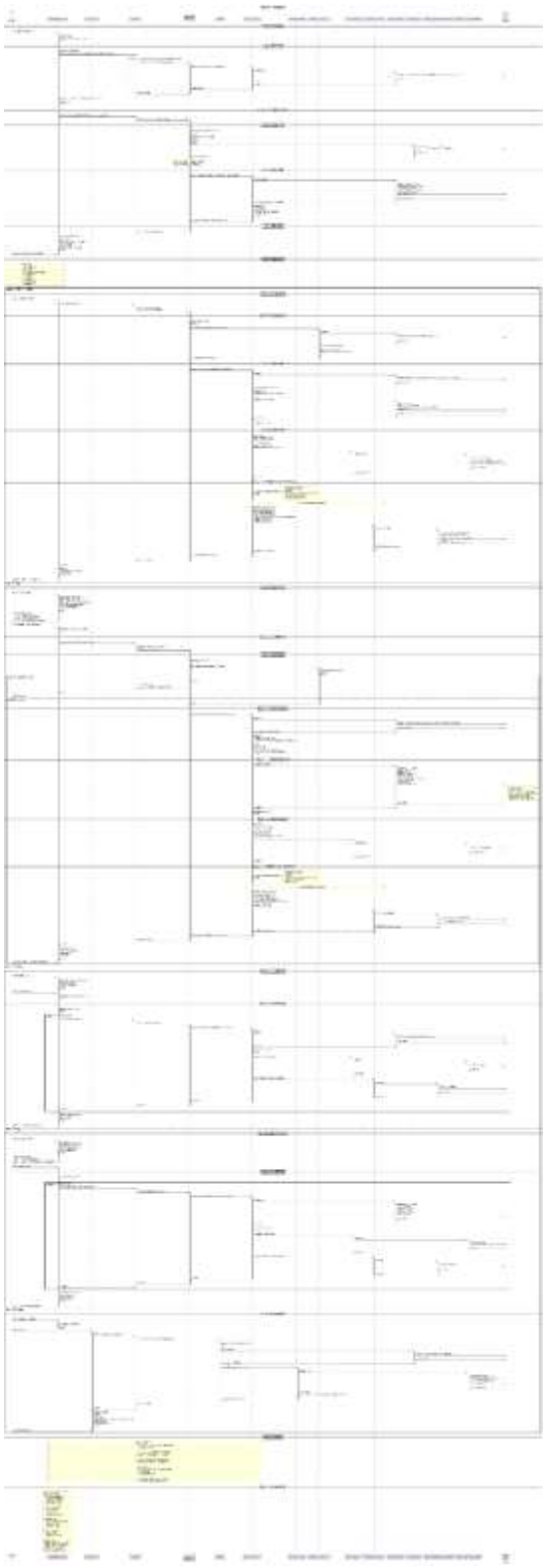


5.5 系統管理

5_1: 使用者管理

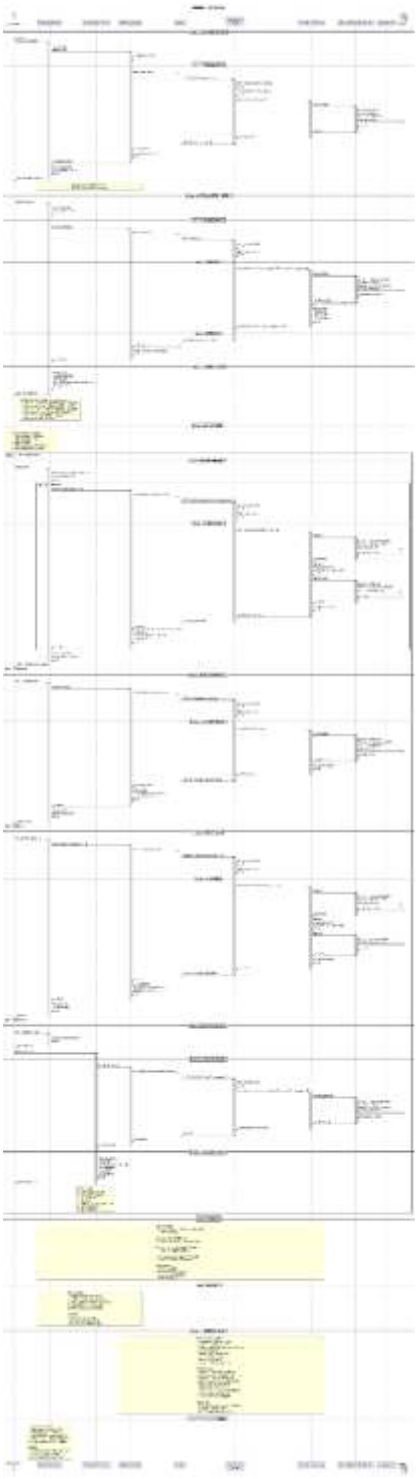


5_2: 資料審核

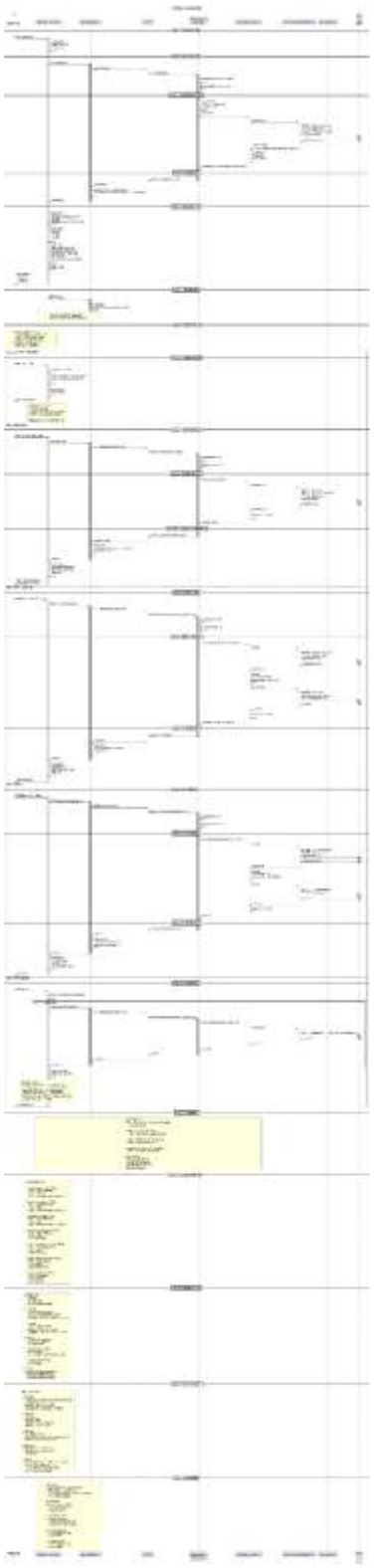


5.6 通知中心

6_1: 通知瀏覽

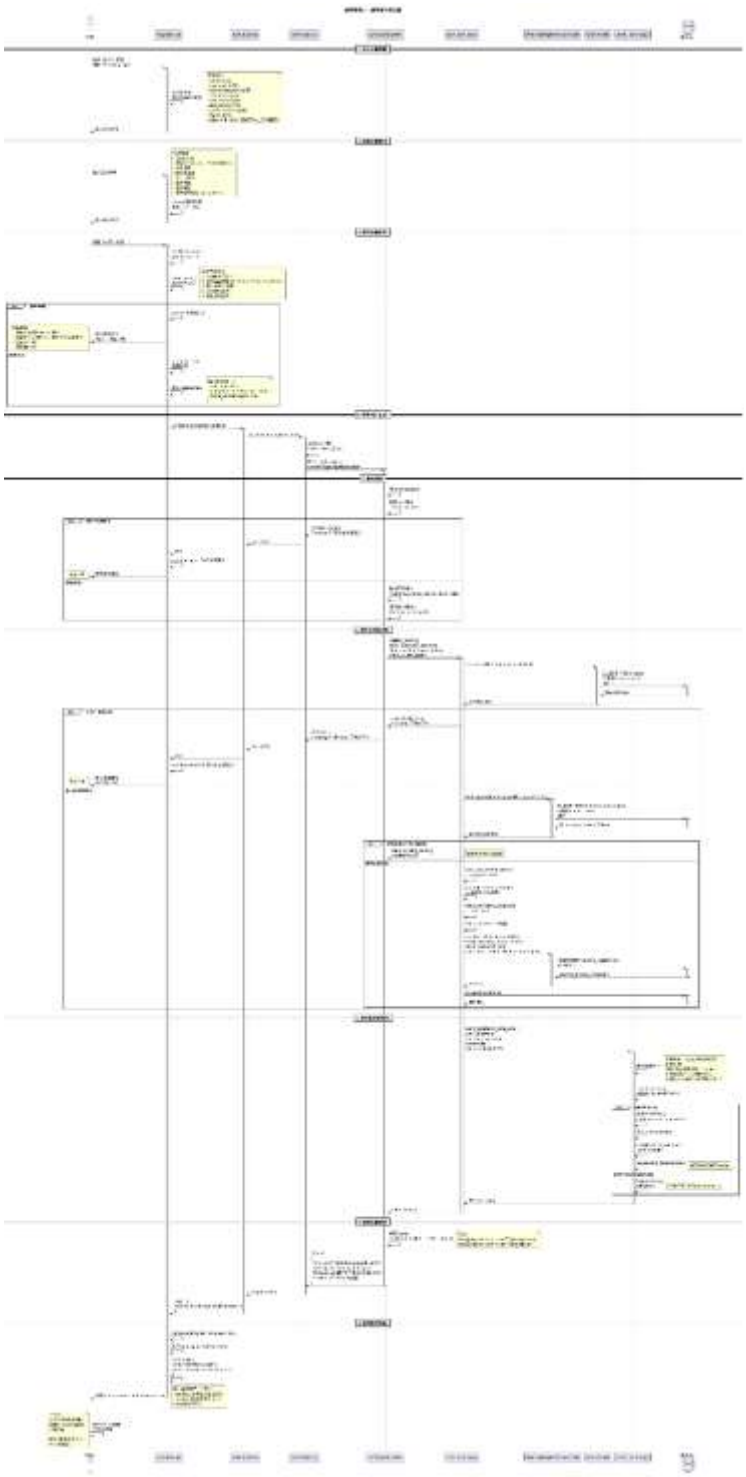


6_2: 通知管理

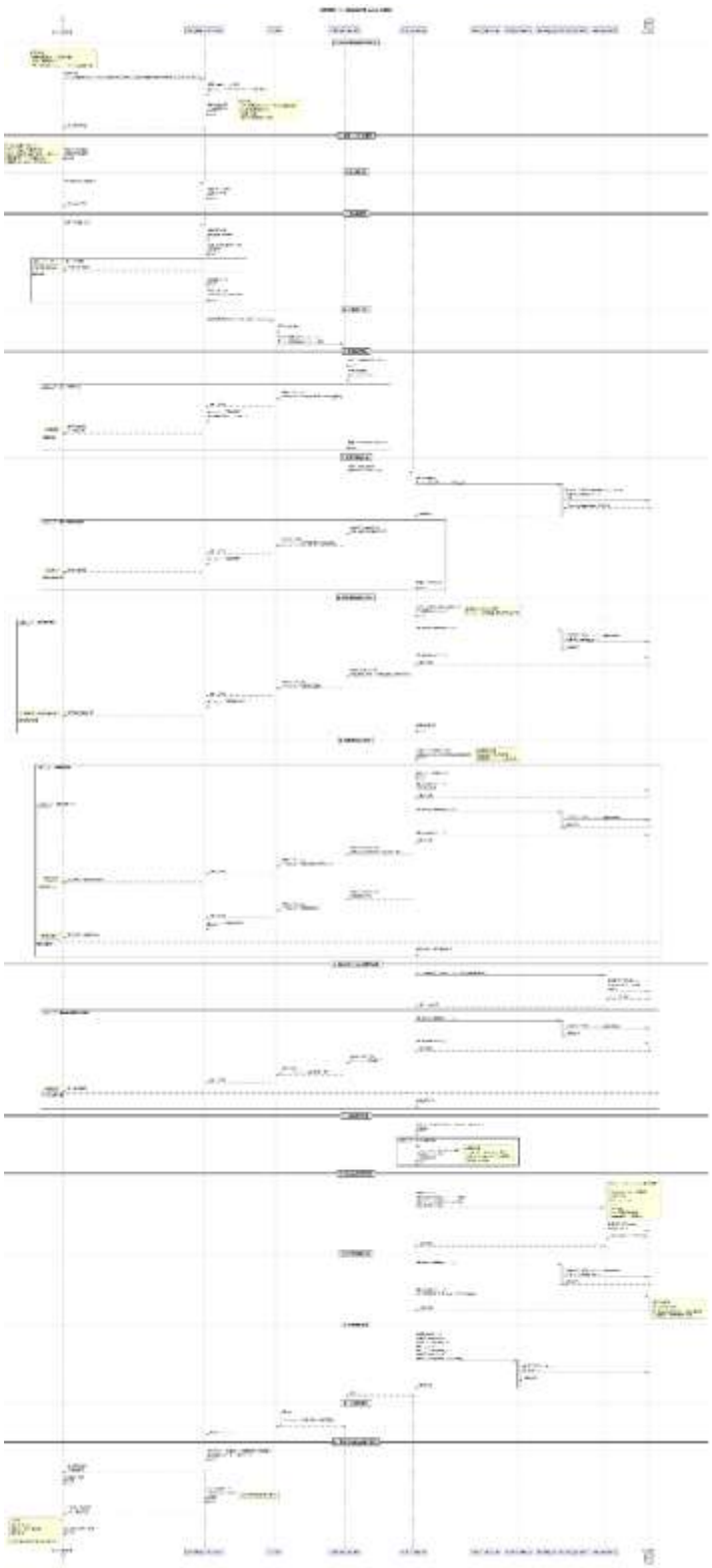


5.7 登入註冊

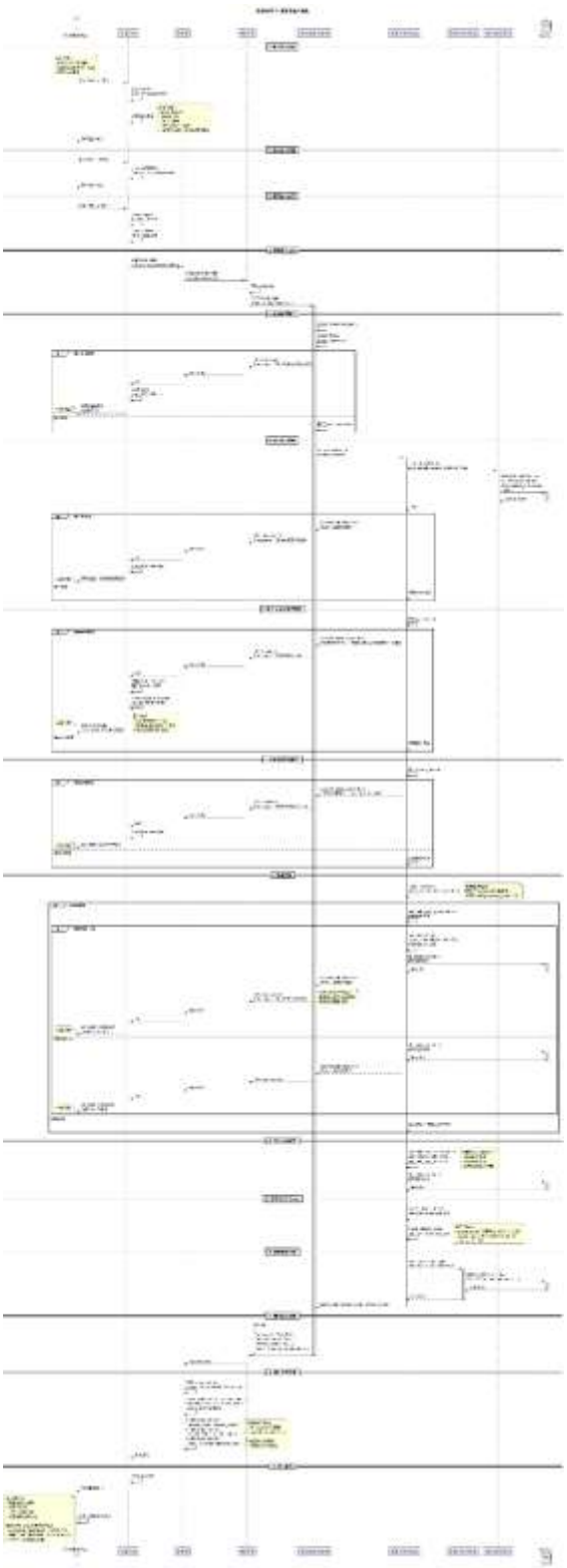
7_1: 使用者註冊



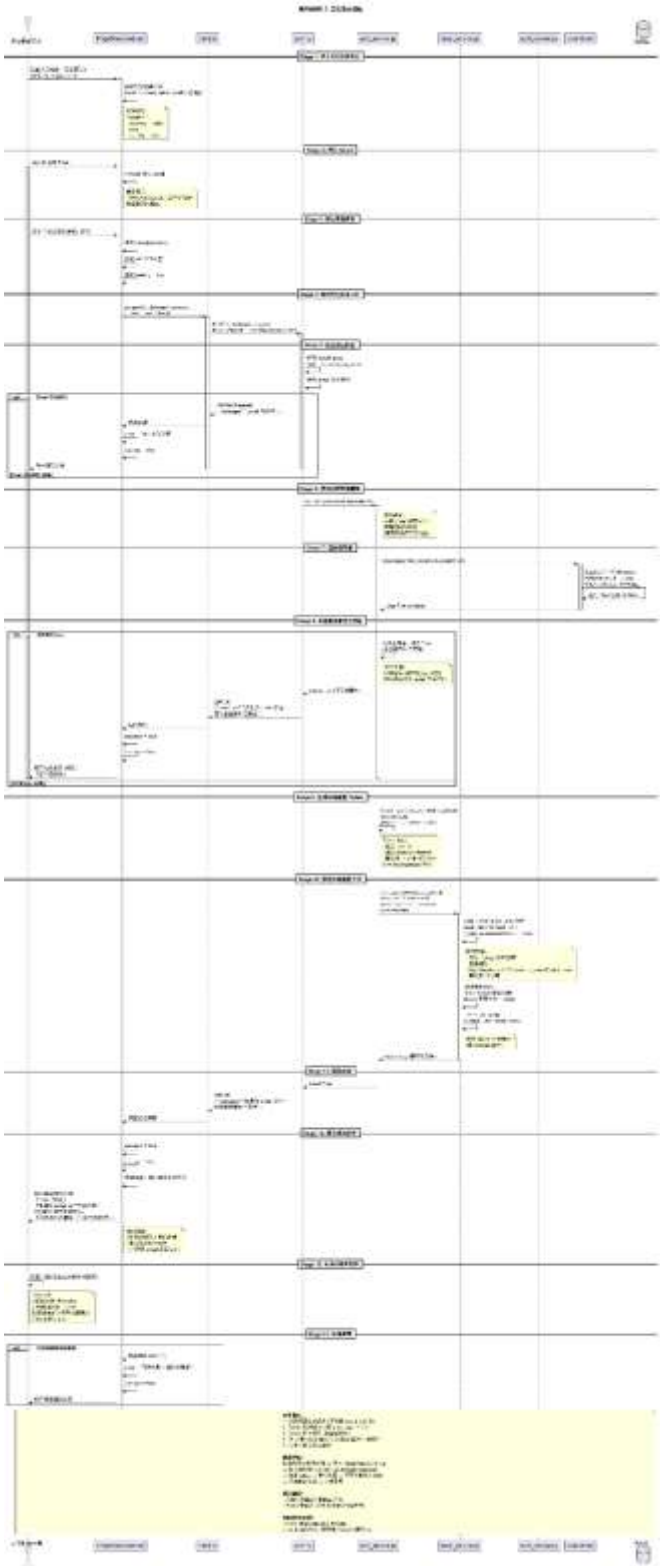
7_1_1: 使用者註冊（帳號驗證）



7_2: 使用者登入



7_3: 密碼管理



7. 資料驗證規則

7.1 後端驗證（Python/Flask）

動物資料驗證:

```python
# animal_service.py
def validate_animal_data(data):
    errors = []

    # 必填欄位
    if not data.get('species'):
        errors.append('物種為必填')

    # 年齡邏輯檢查
    if data.get('dob'):
        dob = datetime.strptime(data['dob'], '%Y-%m-%d').date()
        if dob > date.today():
            errors.append('出生日期不能晚於今天')

    # 狀態轉換驗證
    if data.get('status') == 'PUBLISHED':
        if not data.get('name') or not data.get('description'):
            errors.append('發布需要完整的名稱與描述')

    if errors:
        raise ValidationError('; '.join(errors))
```

申請資料驗證:

```python
# application_service.py
def validate_application_data(data):
    required_fields = ['animal_id', 'contact_phone', 'reason']

    for field in required_fields:
        if not data.get(field):
            raise ValidationError(f'{field} 為必填欄位')

    # 電話號碼格式
    if not re.match(r'^09\d{8}$', data['contact_phone']):
        raise ValidationError('電話號碼格式不正確')
```

7.2 前端驗證（TypeScript/Vue）

使用 vee-validate + zod:

```ts
import { z } from 'zod'

// 動物表單驗證 schema
export const animalSchema = z.object({
  name: z.string().min(1, '名稱為必填').max(200),
  species: z.enum(['CAT', 'DOG'], { required_error: '請選擇物種' }),
  sex: z.enum(['MALE', 'FEMALE', 'UNKNOWN']).optional(),
  dob: z.string().optional(),
  description: z.string().max(5000, '描述不可超過 5000 字').optional(),
})

// 申請表單驗證 schema
export const applicationSchema = z.object({
  animal_id: z.number(),
  contact_phone: z.string()
    .regex(/^09\d{8}$/, '請輸入有效的手機號碼'),
  reason: z.string()
    .min(10, '請至少輸入 10 個字的領養理由')
    .max(1000),
  has_experience: z.boolean(),
})
```

---

8. 錯誤處理

8.1 HTTP 狀態碼規範

狀態碼	說明	使用情境
200 OK	成功	GET, PATCH 成功
201 Created	已建立	POST 建立資源成功
400 Bad Request	錯誤請求	驗證失敗、參數錯誤
401 Unauthorized	未認證	缺少或無效的 JWT token
403 Forbidden	無權限	角色權限不足
404 Not Found	找不到資源	資源不存在或已刪除
409 Conflict	衝突	重複註冊、資源衝突
500 Internal Server Error	伺服器錯誤	未預期的系統錯誤

8.2 錯誤回應格式

標準錯誤格式:

```
{
  "message": "錯誤訊息描述",
  "error_code": "VALIDATION_ERROR",
  "details": {
    "field": "email",
    "reason": "Email 格式不正確"
  }
}
```

前端錯誤處理:

```
// api/client.ts
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      // Token 過期，導向登入頁
      router.push('/login')
    } else if (error.response?.status === 403) {
      // 權限不足
      showError('您沒有執行此操作的權限')
    }
    return Promise.reject(error)
  }
)
```

---

9. 並發控制

9.1 Optimistic Locking

使用 version 欄位防止並發更新衝突

Application 模型:

```
class Application(db.Model):
    version = db.Column(db.Integer, default=1, nullable=False)
```

更新邏輯:

```
def review_application(application_id, action, version):
    application = Application.query.filter_by(
        application_id=application_id
    ).with_for_update().first()

    # 版本檢查
    if application.version != version:
        raise ConflictError('申請已被他人更新，請重新整理')
```

```
    # 執行更新
    application.status = new_status
    application.version += 1
    db.session.commit()
```

9.2 Idempotency (冪等性)

使用 Idempotency-Key 防止重複提交

申請建立:

```
@applications_bp.route('', methods=['POST'])
@jwt_required()
def create_application():
    idempotency_key = request.headers.get('Idempotency-Key')

    if idempotency_key:
        existing = Application.query.filter_by(
            idempotency_key=idempotency_key
        ).first()

        if existing:
            # 回傳已存在的申請
            return jsonify({
                'message': '申請已存在',
                'application': existing.to_dict()
            }), 200

    # 建立新申請...
```

前端實作:

```
import { v4 as uuidv4 } from 'uuid'

export async function createApplication(data: CreateApplicationData) {
  const idempotencyKey = uuidv4()

  const response = await api.post('/applications', data, {
    headers: {
      'Idempotency-Key': idempotencyKey
    }
  })

  return response.data
}
```

---

10. 效能考量

10.1 資料庫查詢優化

索引策略:

```
-- 常用查詢欄位建立索引
CREATE INDEX idx_animals_status ON animals(status);
CREATE INDEX idx_animals_species ON animals(species);
CREATE INDEX idx_animals_shelter_id ON animals(shelter_id);
CREATE INDEX idx_animals_owner_id ON animals(owner_id);
CREATE INDEX idx_animals_created_at ON animals(created_at);

CREATE INDEX idx_applications_status ON applications(status);
CREATE INDEX idx_applications_applicant_id ON applications(applicant_id);
CREATE INDEX idx_applications_animal_id ON applications(animal_id);

CREATE INDEX idx_notifications_recipient_id_read ON notifications(recipient_id, read);
CREATE INDEX idx_users_email ON users(email);
```

查詢優化範例:

```
# ❌ 不好的做法：N+1 查詢問題
animals = Animal.query.filter_by(status='PUBLISHED').all()
for animal in animals:
    print(animal.images)  # 每次都觸發一次查詢

# ✅ 好的做法：使用 joinedload 預載關聯
from sqlalchemy.orm import joinedload

animals = Animal.query.filter_by(status='PUBLISHED')\
    .options(joinedload(Animal.images))\
    .options(joinedload(Animal.owner))\
    .all()
```

分頁查詢:

```
def list_animals_paginated(page=1, per_page=20):
    # 限制 per_page 最大值
    per_page = min(per_page, 100)

    pagination = Animal.query.filter_by(
        status=AnimalStatus.PUBLISHED,
        deleted_at=None
    ).order_by(
        Animal.created_at.desc()
    ).paginate(
        page=page,
        per_page=per_page,
```

```
        error_out=False
    )

    return {
        'animals': [a.to_dict() for a in pagination.items],
        'total': pagination.total,
        'pages': pagination.pages,
        'page': page,
        'per_page': per_page
    }
```

10.2 快取策略

前端快取（@tanstack/vue-query）：

```
// 動物列表 - 快取 5 分鐘
const { data: animals } = useQuery({
  queryKey: ['animals', filters],
  queryFn: () => getAnimals(filters),
  staleTime: 5 * 60 * 1000, // 5 分鐘內視為新鮮
  cacheTime: 10 * 60 * 1000 // 10 分鐘後清除快取
})

// 動物詳情 - 快取 10 分鐘
const { data: animal } = useQuery({
  queryKey: ['animal', animalId],
  queryFn: () => getAnimal(animalId),
  staleTime: 10 * 60 * 1000
})

// 通知數量 - 頻繁更新
const { data: unreadCount } = useQuery({
  queryKey: ['notifications', 'unread-count'],
  queryFn: getUnreadNotificationCount,
  refetchInterval: 10000, // 每 10 秒重新整理
  staleTime: 5000
})
```

後端快取（Redis - 未來擴充）：

```
# 快取熱門動物列表
@cache.memoize(timeout=300) # 5 分鐘
def get_featured_animals():
    return Animal.query.filter_by(
        status=AnimalStatus.PUBLISHED,
        featured=True
    ).limit(10).all()
```

```python
# 快取收容所資訊
@cache.memoize(timeout=3600)  # 1 小時
def get_shelter_info(shelter_id):
    return Shelter.query.get(shelter_id)
```

靜態資源快取:

```
# Nginx 設定
location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
}
```

10.3 圖片處理與 CDN

圖片上傳優化:

```typescript
// 前端壓縮圖片
async function compressImage(file: File): Promise<Blob> {
  return new Promise((resolve, reject) => {
    const reader = new FileReader()
    reader.onload = (e) => {
      const img = new Image()
      img.onload = () => {
        const canvas = document.createElement('canvas')
        const ctx = canvas.getContext('2d')

        // 限制最大寬度 1920px
        const maxWidth = 1920
        const scale = Math.min(1, maxWidth / img.width)

        canvas.width = img.width * scale
        canvas.height = img.height * scale

        ctx.drawImage(img, 0, 0, canvas.width, canvas.height)

        canvas.toBlob((blob) => {
          resolve(blob)
        }, 'image/jpeg', 0.85)  // 85% 品質
      }
      img.src = e.target.result as string
    }
    reader.readAsDataURL(file)
  })
}
```

MinIO + CDN 架構:

```
使用者 → Nginx (CDN) → MinIO (Object Storage)
          ↓
       快取層 (304 Not Modified)
```

圖片 URL 生成:

```python
def get_image_url(storage_key):
    # 開發環境：直接使用 MinIO URL
    if Config.ENV == 'development':
        return f"{Config.MINIO_PUBLIC_ENDPOINT}/{Config.MINIO_BUCKET}/{storage_key}"

    # 生產環境：使用 CDN URL
    return f"{Config.CDN_URL}/{storage_key}"
```

10.4 非同步處理

背景任務類型: 1. **Email 發送** - 立即執行，失敗可重試 2. **批次匯入** - 長時間執行，回傳 job_id 3. **報表生成** - 長時間執行，完成後發通知 4. **圖片處理** - 縮圖生成、格式轉換

Celery 設定:

```python
# celery.py
from celery import Celery

celery = Celery('app')
celery.config_from_object('config.CeleryConfig')

# 定義任務佇列
celery.conf.task_routes = {
    'app.tasks.email_tasks.*': {'queue': 'emails'},
    'app.tasks.animal_tasks.*': {'queue': 'animals'},
    'app.tasks.report_tasks.*': {'queue': 'reports'},
}

# 重試策略
celery.conf.task_annotations = {
    'app.tasks.email_tasks.send_email': {
        'rate_limit': '100/m', # 每分鐘最多 100 封
        'max_retries': 3,
        'default_retry_delay': 60 # 失敗後 60 秒重試
    }
}
```

任務範例:

```python
# email_tasks.py
@celery.task(bind=True, max_retries=3)
def send_notification_email(self, notification_id):
    try:
        notification = Notification.query.get(notification_id)
        recipient = notification.recipient

        send_email(
            to=recipient.email,
            subject=f'新通知：{notification.type}',
            template='notification.html',
            context={'notification': notification}
        )

    except SMTPException as e:
        # 暫時性錯誤，重試
        raise self.retry(exc=e, countdown=60)

    except Exception as e:
        # 永久性錯誤，記錄但不重試
        logger.error(f'Failed to send email for notification {notification_id}: {e}')
```

前端輪詢 vs WebSocket:

```javascript
// 方案一：輪詢（目前實作）
const pollInterval = setInterval(async () => {
  const job = await getJob(jobId)
  if (job.status === 'SUCCEEDED' || job.status === 'FAILED') {
    clearInterval(pollInterval)
    handleJobComplete(job)
  }
}, 5000)

// 方案二：WebSocket（未來擴充）
const socket = new WebSocket('ws://api.example.com/ws')
socket.addEventListener('message', (event) => {
  const data = JSON.parse(event.data)
  if (data.type === 'job_update' && data.job_id === jobId) {
    handleJobUpdate(data)
  }
})
```

10.5 Rate Limiting (速率限制)

後端 Rate Limiting:

```python
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

# 針對特定端點設定
@animals_bp.route('', methods=['POST'])
@limiter.limit("10 per hour") # 每小時最多建立 10 個動物
@jwt_required()
def create_animal():
    pass

@applications_bp.route('', methods=['POST'])
@limiter.limit("5 per hour") # 每小時最多提交 5 個申請
@jwt_required()
def create_application():
    pass
```

---

11. 安全性考量

11.1 認證與授權

JWT Token 管理:

```python
# config.py
JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY')
JWT_ACCESS_TOKEN_EXPIRES = timedelta(hours=1)
JWT_REFRESH_TOKEN_EXPIRES = timedelta(days=30)
```

密碼安全:

```python
from werkzeug.security import generate_password_hash, check_password_hash

# 註冊時
password_hash = generate_password_hash(password, method='pbkdf2:sha256')

# 登入時
if not check_password_hash(user.password_hash, password):
    raise UnauthorizedError('密碼錯誤')
```

RBAC 權限檢查:

```python
def check_permission(user, action, resource):
    # 管理員擁有所有權限
    if user.role == UserRole.ADMIN:
        return True

    # 檢查資源擁有權
    if action in ['UPDATE', 'DELETE']:
        if resource.owner_id == user.user_id:
            return True
        if hasattr(resource, 'created_by') and resource.created_by == user.user_id:
            return True

    # 收容所成員權限
    if user.role == UserRole.SHELTER_MEMBER:
        if hasattr(resource, 'shelter_id') and resource.shelter_id == user.primary_shelter_id:
            return True

    return False
```

11.2 輸入驗證

SQL Injection 防護:

```python
# ✅ 使用 ORM，自動處理參數化查詢
animals = Animal.query.filter_by(name=user_input).all()

# ❌ 避免直接拼接 SQL
# query = f"SELECT * FROM animals WHERE name = '{user_input}'" # 危險！
```

XSS 防護:

```vue
// 前端：自動轉義（Vue 預設行為）
<template>
  <div>{{ animal.description }}</div>  // 自動轉義
  <div v-html="sanitizedHtml"></div>  // 需手動淨化
</template>

// 淨化 HTML
import DOMPurify from 'dompurify'

const sanitizedHtml = computed(() => {
  return DOMPurify.sanitize(animal.value.description)
})
```

CSRF 防護:

```python
# Flask-WTF CSRF Protection
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect(app)

# JWT 不需要 CSRF token（存在 localStorage）
# 但 Cookie-based auth 需要
```

11.3 檔案上傳安全

檔案類型驗證:

```python
ALLOWED_IMAGE_TYPES = {'image/jpeg', 'image/png', 'image/gif', 'image/webp'}
ALLOWED_DOCUMENT_TYPES = {'application/pdf'}
MAX_FILE_SIZE = 10 * 1024 * 1024  # 10MB

def validate_file(file, allowed_types):
    # 檢查 MIME type
    if file.mimetype not in allowed_types:
        raise ValidationError(f'不支援的檔案類型: {file.mimetype}')

    # 檢查檔案大小
    file.seek(0, 2) # 移到檔案末尾
    size = file.tell()
    file.seek(0) # 回到開頭

    if size > MAX_FILE_SIZE:
        raise ValidationError(f'檔案過大: {size} bytes')

    # 檢查檔案內容（magic bytes）
    header = file.read(512)
    file.seek(0)

    if not is_valid_image(header):
        raise ValidationError('檔案內容不符合宣告的類型')
```

安全的檔名處理:

```python
import uuid
import os

def generate_safe_filename(original_filename):
    # 取得副檔名
    ext = os.path.splitext(original_filename)[1].lower()

    # 生成 UUID 檔名
```

```
    safe_name = f"{uuid.uuid4()}{ext}"

    return safe_name
```

11.4 敏感資料處理

環境變數管理:

```
# .env (不應提交到 git)
DATABASE_URL=postgresql://user:pass@localhost/db
JWT_SECRET_KEY=your-secret-key-here
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    DATABASE_URL = os.getenv('DATABASE_URL')
    JWT_SECRET_KEY = os.getenv('JWT_SECRET_KEY')
```

敏感欄位遮罩:

```
def to_dict(self, include_sensitive=False):
    data = {
        'user_id': self.user_id,
        'email': self.email,
        'username': self.username,
    }

    if include_sensitive:
        data.update({
            'phone_number': self.phone_number,
            'address': self.address,
        })
    else:
        # 遮罩敏感資料
        data['phone_number'] = self._mask_phone(self.phone_number)

    return data

def _mask_phone(self, phone):
    if not phone or len(phone) < 4:
        return '****'
    return phone[:4] + '****'
```

附錄 A: 完整 API 端點列表

Authentication

方法	端點	描述	認證	權限
POST	/auth/register	使用者註冊	No	-
POST	/auth/login	使用者登入	No	-
POST	/auth/refresh	刷新 access token	Yes (Refresh)	-
GET	/auth/me	取得當前使用者資訊	Yes	-
POST	/auth/logout	登出	Yes	-
GET	/auth/verify	Email 驗證	No	-
POST	/auth/forgot-password	忘記密碼	No	-
POST	/auth/reset-password	重設密碼	No	-

Animals

方法	端點	描述	認證	權限
GET	/animals	動物列表（公開）	Optional	-
GET	/animals/:id	動物詳情	Optional	-
POST	/animals	建立動物	Yes	GENERAL_MEMBER, SHELTER_MEMBER, ADMIN
PATCH	/animals/:id	更新動物	Yes	Owner, Admin
DELETE	/animals/:id	刪除動物（軟刪除）	Yes	Owner, Admin
POST	/animals/:id/images	新增動物圖片	Yes	Owner, Admin
DELETE	/animals/:id/images/:image_id	刪除動物圖片	Yes	Owner, Admin
PATCH	/animals/:id/images/reorder	重新排序圖片	Yes	Owner, Admin
POST	/animals/:id/submit	提交審核	Yes	Owner
POST	/animals/:id/publish	發布動物	Yes	Admin

方法	端點	描述	認證	權限
POST	/animals/:id/retire	下架動物	Yes	Owner, Admin
POST	/animals/:id/reject	拒絕動物	Yes	Admin

Applications

方法	端點	描述	認證	權限
GET	/applications	申請列表	Yes	-
GET	/applications/:id	申請詳情	Yes	Applicant, Owner, Admin
POST	/applications	建立申請	Yes	GENERAL_MEMBER
POST	/applications/:id/review	審核申請	Yes	Owner, Admin
POST	/applications/:id/assign	指派審核人	Yes	Owner, Admin
POST	/applications/:id/withdraw	撤回申請	Yes	Applicant

Medical Records

方法	端點	描述	認證	權限
GET	/medical-records/animals	可管理的動物列表	Yes	-
GET	/medical-records/animals/:id/medical-records	動物醫療紀錄列表	Optional	-
POST	/medical-records/animals/:id/medical-records	建立醫療紀錄	Yes	Owner, Admin
PATCH	/medical-records/:id	更新醫療紀錄	Yes	Creator, Admin
DELETE	/medical-records/:id	刪除醫療紀錄	Yes	Creator, Admin

Shelters

方法	端點	描述	認證	權限
GET	/shelters	收容所列表（公開）	No	-
GET	/shelters/:id	收容所詳情	No	-
POST	/shelters	建立收容所	Yes	SHELTER_MEMBER,

方法	端點	描述	認證	權限
				ADMIN
PATCH	/shelters/:id	更新收容所	Yes	Manager, Admin
POST	/shelters/:id/animals/batch	批次匯入動物	Yes	Manager, Admin
GET	/shelters/:id/animals	收容所動物列表	No	-
GET	/shelters/:id/statistics	收容所統計資料	No	-

Notifications

方法	端點	描述	認證	權限
GET	/notifications	通知列表	Yes	-
GET	/notifications/unread-count	未讀通知數量	Yes	-
POST	/notifications/:id/read	標記已讀	Yes	Recipient
POST	/notifications/read-all	全部標記已讀	Yes	-
DELETE	/notifications/:id	刪除通知	Yes	Recipient

Jobs

方法	端點	描述	認證	權限
GET	/jobs	任務列表	Yes	-
GET	/jobs/:id	任務詳情	Yes	Creator, Admin

Uploads

方法	端點	描述	認證	權限
POST	/uploads/direct	直接上傳檔案	Yes	-
POST	/uploads/presign	取得預簽名 URL	Yes	-
POST	/attachments	建立附件記錄	Yes	-

Users

方法	端點	描述	認證	權限
GET	/users/:id	使用者公開資料	No	-
PATCH	/users/:id	更新使用者資料	Yes	Self, Admin
POST	/users/:id/change-password	變更密碼	Yes	Self

Admin

方法	端點	描述	認證	權限
GET	/admin/users	使用者管理列表	Yes	ADMIN
POST	/admin/users/:id/update-role	更新使用者角色	Yes	ADMIN
POST	/admin/users/:id/lock	鎖定使用者	Yes	ADMIN
POST	/admin/users/:id/unlock	解鎖使用者	Yes	ADMIN
GET	/admin/audit-logs	稽核日誌	Yes	ADMIN
GET	/admin/statistics	系統統計資料	Yes	ADMIN

總計: 60+ API 端點
